ALMA MATER STUDIORUM - UNIVERSITÀ DI BOLOGNA

Dipartimento di Informatica, scienze e ingegneria

Laurea in Ingegneria Informatica

Valutazione delle prestazioni di infrastrutture Mesh in Kubernetes

RELATORE
Antonio Corradi

CORRELATORE
Andrea Sabbioni

CANDIDATO
Ulisse Steccanella

ANNO ACCADEMICO 2024/2025

Indice

Introduzione 4
Capitolo 1: Cloud Computing 5
1.1 Le componenti del Cloud Computing 5
1.2 Tipologie di servizi del Cloud Computing 6
1.3 Macchine virtuali e container 7
1.4 Funzionamento di un container 9
Capitolo 2: Microservizi 10
2.1 Vantaggi dell'architettura a microservizi rispetto a quella monolitica 11
2.2 Sicurezza nell'utilizzo di microservizi: Zero Trust e SPIFFE 12
2.3 Orchestrazione 14
Capitolo 3: Kubernetes 15
3.1 I componenti di Kubernetes 16
3.2 Addon 18
3.3 Oggetti in Kubernetes 19
3.4 Custom Resources 20
3.5 Kubernetes Self-Healing 22
Capitolo 4: Service Mesh 24
4.1 Envoy 25
4.2 Tecnologie per Service Mesh: Istio e Network Service Mesh 26
Capitolo 5: Istio 28
5.1 Architettura 28
5.2 Esempi di utilizzo di Istio 30
5.3 Gestione del traffico 31
5.4 Virtual Services 32
5.5 Destination Rules 33
5.6 Gateways 34
5.7 Service Entries 36
5.8 Osservabilità 37

5.9 Sicurezza 37
5.10 Health Check 38
5.11 Istio per sistemi legacy 39
Capitolo 6: Network Service Mesh 41
6.1 Architettura 42
6.2 Network Service API 43
6.3 Componenti 45
6.4 Esempio di caso d'uso di NSM 46
6.5 Instaurazione di una connessione tra Pod 46
6.6 Tipi di configurazioni della connessione tra Pod 47
6.7 Osservabilità 49
6.8 Sicurezza 50
6.9 Healing e resilienza 51
6.10 Cooperazione tra Service Mesh e NSM 51
6.11 Confronto tra Istio e NSM 52
Capitolo 7: Test di confronto tra K3s semplice, Istio e NSM 57
7.1 K3s 59
7.2 Consumo risorse dei Control Plane di Istio e NSM 60
7.3 Tempo di avvio di ogni configurazione 61
7.4 Test richieste HTTP 62
7.5 Test richieste TCP 65
7.6 Test richieste gRPC 68
7.7 Test traffico variabile 73
7.8 Test simulazione fallimento del Data Plane 83
Conclusioni 85
Bibliografia 87

Introduzione

Negli ultimi anni il paradigma del Cloud Computing ha trasformato radicalmente il modo in cui vengono progettate, distribuite e gestite le applicazioni. Le risorse sono diventate scalabili e flessibili e si sono diffuse architetture a microservizi e modelli di deployment dinamici. Quest'ultime sono gestite da piattaforme di orchestrazione avanzate come Kubernetes, che oggi è diventato standard de facto per la gestione di applicazioni containerizzate in ambienti Cloud Native.
Con la crescente complessità degli ecosistemi distribuiti è emersa l'esigenza di soluzioni capaci di gestire in modo trasparente aspetti fondamentali come comunicazione, sicurezza, osservabilità e resilienza dei servizi. In tale contesto si colloca il paradigma Service Mesh: si tratta di un'infrastruttura che gestisce la comunicazione tra microservizi, separando la logica di rete (come discovery, bilanciamento, sicurezza, osservabilità) dal codice applicativo, tramite un livello di astrazione dedicato che semplifica e controlla le interazioni.
Tra le implementazioni più note di Service Mesh si distinguono Istio, oggi molto utilizzato per la gestione avanzata del traffico, e Network Service Mesh (NSM), che si focalizza sull'integrazione di servizi di rete complessi all'interno di Kubernetes. Entrambi perseguano l'obiettivo di semplificare e potenziare la comunicazione tra servizi, ma adottano approcci architetturali e filosofie differenti.
Questa tesi si propone di analizzare e confrontare Istio e Network Service Mesh, mettendone in evidenza tecnologie usate, funzionalità e integrazione con il cluster. Nella parte sperimentale sono stati svolti una serie di test mirati a misurare le prestazioni e l'utilizzo delle risorse delle due tecnologie, al fine di fornire una valutazione oggettiva e supportata da dati. L'obiettivo è offrire un contributo utile a comprendere le implicazioni pratiche della scelta di una tecnologia piuttosto che l'altra, in relazione a diversi scenari d'uso in Kubernetes.

Capitolo 1: Cloud Computing

Con **Cloud Computing** si intende l'erogazione di servizi offerti su richiesta (on-demand) da un fornitore a un utente finale attraverso la rete internet (come l'archiviazione, l'elaborazione o la trasmissione di dati), a partire da un insieme di risorse preesistenti, configurabili e disponibili in remoto sotto forma di architettura distribuita [1].
I vantaggi del Cloud Computing possono essere [1]:

- Economici: le organizzazioni possono esternalizzare parzialmente o totalmente i costi e le complessità legate all'acquisto, installazione, configurazione e gestione di mainframe e altre infrastrutture on-premise.
- Maggiore velocità e agilità operativa: incremento della rapidità di risposta e dell'agilità organizzativa. Le imprese possono avviare applicazioni enterprise in pochi minuti, evitando le tempistiche per inoltrare richieste ai reparti IT, acquistare e configurare hardware dedicato e installare software. Questa caratteristica favorisce in particolare i team DevOps e di sviluppo, che possono disporre autonomamente di software e infrastrutture di supporto, accelerando l'innovazione.
- Scalabilità illimitata: il Cloud Computing offre elasticità e provisioning self-service, permettendo di adeguare la capacità di calcolo in base alle effettive esigenze. In questo modo, le aziende evitano di acquistare infrastrutture in eccesso che resterebbero inutilizzate nei periodi di bassa attività, potendo invece espanderle o ridurle dinamicamente in risposta a variazioni di traffico. Inoltre, grazie alle reti globali dei cloud provider, è possibile distribuire le applicazioni in prossimità degli utenti finali in tutto il mondo.
- Valore strategico potenziato: infine, il Cloud Computing garantisce alle organizzazioni l'accesso a tecnologie avanzate e innovazioni costanti, rendendole più competitive nel mercato. Nel settore manifatturiero, per esempio, il cloud favorisce la collaborazione e l'utilizzo di software per monitorare dati in tempo reale lungo l'intera catena logistica e di approvvigionamento.

Nel seguente capitolo verranno illustrate le componenti del Cloud Computing, le tipologie di servizi che può offrire e alcuni strumenti software comunemente utilizzati in contesti cloud.

1.1 Le componenti del Cloud Computing

I **data center** sono strutture fisiche in cui infrastrutture IT ospitano la creazione, esecuzione, fruizione e archiviazione di applicazioni e servizi.

I **Cloud Service Provider** (CSP) possiedono e gestiscono data center remoti che ospitano server fisici o bare metal, sistemi di storage cloud e altre infrastrutture hardware fisiche che costituiscono la base sottostante e forniscono le fondamenta fisiche per il Cloud Computing [1].

Con **networking capabilities** si intende le funzioni e le tecnologie che consentono a dispositivi, programmi informatici e persone di connettersi, comunicare e scambiare risorse nella rete. Per esempio, una wide-area network (WAN) connette gli utenti al back-end delle applicazioni, ovvero a data center e servizi cloud [1].

La **virtualizzazione** serve a creare risorse informatiche virtuali (server, storage, reti) su hardware fisico condiviso. Per esempio, più macchine virtuali possono girare su un singolo server fisico, si possono scalare le risorse in modo rapido e flessibile, si gestiscono e isolano facilmente ambienti diversi (ad esempio, test, produzione) [1].

Un **sistema legacy** è un sistema informatico oppure un'applicazione software obsoleta ma ancora in uso all'interno di un'organizzazione. Utilizzano tecnologie non più supportate, pattern architetturali inefficienti e sono difficili da integrare in un sistema cloud. Spesso questi sistemi vengono ancora usati perché svolgono funzioni critiche e non si riesce a trovare le modalità o le risorse economiche per sostituirli. Il Cloud Computing si occupa anche di integrare i sistemi legacy in sistemi cloud moderni, in modo efficace e sicuro.
I sistemi di Service Mesh di cui tratta questa tesi offrono funzionalità differenti per interagire con un sistema legacy, l'argomento sarà affrontato nei capitoli successivi.

1.2 Tipologie di servizi del Cloud Computing

I Cloud Service Provider possono fornire e gestire le risorse IT in diversi modi. Di seguito verranno illustrati i principali modelli di Cloud Computing.

Infrastructure as a Service (IaaS): il provider cloud offre la rete, lo storage e le macchine virtuali on demand. In questo caso, i clienti sono responsabili di tutti i livelli sopra la macchina virtuale, insieme alla configurazione del sistema operativo e ai suoi aggiornamenti. Lo scopo di questa soluzione è dare ai clienti il massimo livello di controllo e di responsabilità sulla configurazione e sull'utilizzo delle risorse virtualizzate. Esempio: Amazon EC2 [1].

Platform as a Service (Paas): questo modello rappresenta un ambiente predefinito, solitamente composto da risorse IT già distribuite e configurate. Il

provider cloud si occupa di mantenere il sistema operativo e fornisce soluzioni middleware come database, messaggistica aziendale ed e runtime containers. In questo modello, i clienti si concentrano sulle funzionalità dell'applicazione e sull'automazione della loro distribuzione. Esempio: Google App Engine [1].

Software as a Service (Saas): il provider cloud gestisce l'hardware e il software offrendoli come prodotto finale o come servizio generico. Viene utilizzato per rendere disponibile un servizio cloud riutilizzabile su larga scala a un vasto numero di clienti. Esempio: Office 365 [1].

Serverless computing: modello cloud in cui tutta la gestione dell'infrastruttura (come configurare server, scalare risorse, fare aggiornamenti) è a carico del provider. In questo modo, gli sviluppatori possono dedicarsi unicamente a scrivere il codice e a sviluppare la logica dell'applicazione, senza preoccuparsi dei server. Con il modello serverless, il codice viene eseguito solo quando serve, cioè in risposta a specifiche richieste, e le risorse vengono aumentate o ridotte in automatico a seconda del carico di lavoro. I clienti pagano solo quando l'applicazione viene effettivamente eseguita e non sostengono costi per server o capacità che rimangono inutilizzati [1].

	On-premises	IaaS	PaaS	SaaS
Applicazione	Cliente	Cliente	Cliente	CSP
Dati	Cliente	Cliente	Cliente	CSP
Runtime	Cliente	Cliente	CSP	CSP
Middleware	Cliente	Cliente	CSP	CSP
Sistema Operativo	Cliente	Cliente	CSP	CSP
Virtualizzazione	Cliente	CSP	CSP	CSP
Server/Storage	Cliente	CSP	CSP	CSP
Rete	Cliente	CSP	CSP	CSP

Tabella 1.1: Responsabilità del Cliente e del CSP (Cloud Service Provider) per ogni modello

1.3 Macchine virtuali e container

La **virtualizzazione** di un sistema operativo (SO) consiste nell'eseguire più sistemi operativi su una singola macchina fisica, utilizzando un software di virtualizzazione. Questo software crea un ambiente virtuale, detto "macchina virtuale" (MV), che emula le funzionalità di una macchina fisica (astrazione dell'hardware dell'host). Ogni MV può eseguire un proprio sistema operativo, indipendentemente dagli altri.
In genere un singolo server ospita più macchine virtuali, con un hypervisor che funge da software leggero tra l'host fisico e le MV. L'hypervisor gestisce l'accesso alle risorse in modo efficiente e consente alle macchine virtuali di operare come server distinti, offrendo al tempo stesso più agilità e flessibilità. Quando un utente

invia alla MV un'istruzione che richiede ulteriori risorse dell'ambiente fisico, l'hypervisor inoltra la richiesta al sistema fisico ed esegue il caching delle modifiche [2].

Un **container** è un'astrazione che definisce un contenitore per applicazioni software, alle quali fornisce tutto il necessario (codice, esecuzione runtime, strumenti/librerie di sistema ed impostazioni) virtualizzando le risorse fisiche fornite dal SO sottostante. Il container è dipendente dal kernel host per le funzionalità di sistema e utilizza le sue chiamate di sistema (astrazione del SO host). La portabilità di un container è quindi possibile solo in una macchina con un kernel compatibile.
Il disaccoppiamento tra implementazione e ambiente consente di eseguire facilmente il deployment delle applicazioni containerizzate, indipendentemente dall'ambiente di destinazione, che sia un data center privato, il cloud pubblico o il laptop di uno sviluppatore [2].

Entrambi i metodi consentono l'esecuzione di un numero di MV/container in parallelo sulla stessa macchina, in questo modo permettono lo sviluppo e l'esecuzione di applicazioni nell'ambiente desiderato, a prescindere dalla macchina ospitante.
I container si sono affermati rispetto alle MV perché consumano meno risorse (non hanno un proprio kernel e SO), il loro avvio è molto veloce (si tratta di avviare un processo isolato) e sono più portabili (applicazione e dipendenze in un unico oggetto eseguibile). Nel contesto del Cloud Computing, i container rappresentano una tecnologia chiave poiché consentono di racchiudere applicazioni e dipendenze in ambienti isolati, leggeri e facilmente trasportabili. I container sono oggi molto utilizzati per contenere microservizi, operazioni di test, build e deploy automatizzati (DevOps) e ambienti nativi cloud (come Kubernetes). La loro portabilità garantisce che il software si comporti in maniera coerente su differenti infrastrutture, semplificando i processi di sviluppo e distribuzione [2].

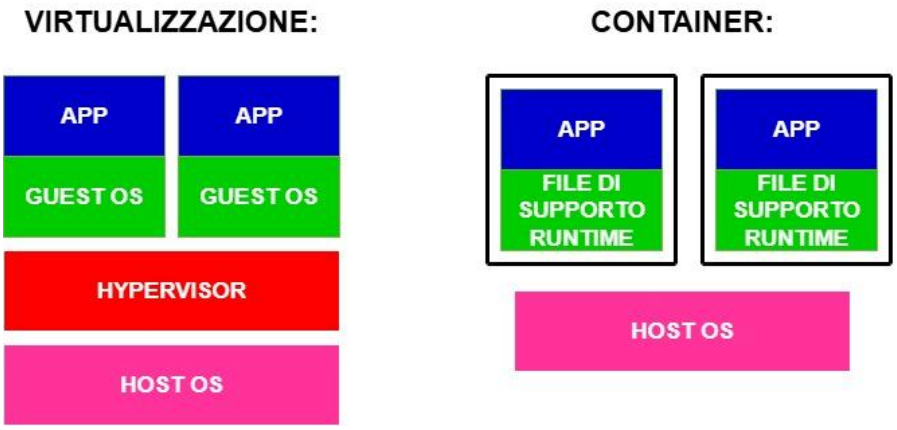

Figura 1.1: Confronto tra virtualizzazione e container

La Figura 1.1 propone un confronto tra virtualizzazione e container indicando le rispettive componenti principali. A sinistra è rappresentato un sistema operativo che utilizza un hypervisor a supporto del funzionamento di due macchine virtuali. A destra è rappresentato un sistema operativo in cui sono in esecuzione due container, con i rispettivi file di supporto runtime.

Nella parte sperimentale di questa tesi sono state usate entrambe queste tecnologie ma con finalità diverse. Sono state utilizzate due macchine virtuali come server locali per l'installazione e l'utilizzo di tecnologie Kubernetes.
In queste macchine virtuali, che formano un cluster Kubernetes, sono in esecuzione diversi container che contengono software utile per effettuare test.

1.4 Funzionamento di un container

Un container usufruisce di diverse funzionalità del kernel Linux tra cui: cgroups, namespaces e copy-on-write storage.

I **control groups** (cgroup) permettono di assegnare, monitorare e limitare l'utilizzo delle risorse di sistema (come CPU, memoria, I/O e rete) da parte di un insieme di processi. Questo significa che è possibile definire quote precise di risorse per ciascun container, evitando che un singolo processo monopolizzi l'intero sistema [3].

I **namespaces** forniscono isolamento tra i processi, creando per ogni container una "visione" separata del sistema operativo. Esistono diversi tipi di namespace, ognuno dei quali isola una componente specifica del sistema. Grazie ai namespace, i processi in esecuzione all'interno di un container non hanno visibilità diretta sugli altri processi o risorse dell'host o di altri container [3].

La tecnologia **copy-on-write storage** consente la creazione efficiente di nuovi container a partire da un'immagine base. Quando un container viene avviato, non viene copiata l'intera immagine, ma viene creata una nuova "vista" in scrittura, dove solo le modifiche rispetto all'immagine originaria vengono salvate. Ciò consente tempi di avvio molto rapidi [3].

Nel seguente capitolo è stato introdotto il concetto di Cloud Computing, i suoi contesti di utilizzo e tecnologie usate. Nel prossimo capitolo verrà spiegato come vengono usati i container in un'architettura a microservizi.

Capitolo 2: Microservizi

Negli ultimi anni, con l'evoluzione delle architetture software verso modelli sempre più distribuiti, l'approccio a **microservizi** si è affermato come paradigma dominante nello sviluppo di applicazioni Cloud Native. Questo modello prevede la suddivisione di un'applicazione in una serie di servizi indipendenti, ciascuno dedicato a una specifica funzione, in grado di comunicare tra loro attraverso API leggere. Rispetto allo stile monolitico tradizionale, per cui ogni componente viene generato all'interno di un unico elemento, i microservizi interagiscono per portare a termine le stesse attività. Tale organizzazione offre vantaggi significativi in termini di scalabilità, flessibilità nello sviluppo e resilienza, ma introduce allo stesso tempo nuove complessità, in particolare nella gestione della comunicazione tra i servizi [4]. Sia i microservizi che i container sono componenti indipendenti e stateless. Ogni microservizio viene incapsulato in un container, creando così un ambiente virtualizzato con l'opportunità di scalare orizzontalmente il microservizio solo su richiesta, sfruttando al meglio le risorse [2].

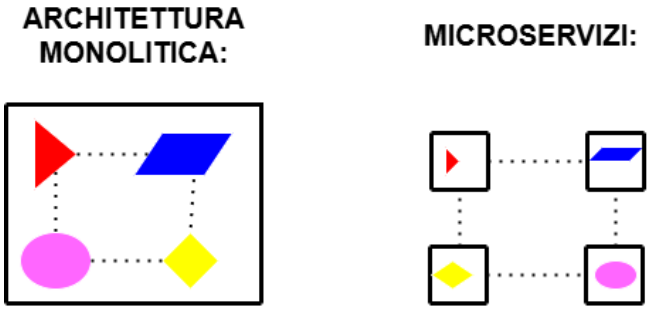

Figura 2.1: Architettura monolitica e a microservizi

In Figura 2.1 è mostrato un confronto tra architettura monolitica e a microservizi. Nella prima le componenti applicative vengono eseguite in un unico blocco, mentre nella seconda le componenti applicative sono divise tra diversi container.

Nel seguente capitolo verranno approfonditi i vantaggi introdotti dall'utilizzo di architetture a microservizi, due soluzioni per usare quest'ultime in sicurezza e infine verrà spiegato il concetto di orchestrazione. L'obiettivo è capire il contesto in cui viene inserita una Service Mesh, che serve proprio per agevolare l'interazione tra microservizi.

2.1 Vantaggi dell'architettura a microservizi rispetto a quella monolitica

Nelle **architetture monolitiche**, tutti i processi dell'applicazione sono strettamente interconnessi e vengono eseguiti come un unico blocco. Questo comporta che, se uno di questi processi registra un aumento di richieste, occorre scalare l'intera applicazione, non solo la parte interessata. Inoltre, introdurre o modificare una funzionalità in un sistema monolitico richiede di ampliare la base di codice complessiva, rendendo più complicata la manutenzione e scoraggiando l'adozione di nuove soluzioni o esperimenti. Questa struttura aumenta anche il rischio di interruzioni del servizio: un malfunzionamento in un singolo processo può compromettere l'intera applicazione, data la stretta dipendenza tra le parti [4].

Al contrario, un'**architettura a microservizi** suddivide l'applicazione in moduli indipendenti, ognuno dei quali gestisce un processo specifico sotto forma di servizio autonomo. Questi servizi comunicano fra loro tramite API leggere e ben definite. Ogni microservizio è progettato per svolgere un'unica funzione legata alle esigenze aziendali e, grazie alla sua autonomia, può essere aggiornato, implementato o scalato singolarmente, rispondendo in modo mirato a eventuali picchi di domanda per quella funzione specifica [4].

Alcuni vantaggi dei microservizi sono [4]:

- Autonomia: ciascun microservizio può essere sviluppato, distribuito, eseguito e ridimensionato senza influenzare il funzionamento degli altri componenti.
- Specializzazione: ciascun microservizio si occupa di un compito specifico. Se gli sviluppatori vogliono aggiungere codice ad un servizio, possono valutare di scomporre lo stesso in più microservizi più piccoli
- Agilità: i tempi di sviluppo sono ridotti visto che si possono creare piccoli team di sviluppo che agiscono in contesti ridotti e ben delineati e lavorano in modo più indipendente e rapido.
- Eliminazione di "single point of failure": un errore nel codice o di configurazione del microservizio comporta un malfunzionamento solo locale, del microservizio stesso. Raramente è compromesso il funzionamento dell'intero sistema. Eventuali servizi instabili possono essere corretti e distribuiti nuovamente senza disturbare il funzionamento dell'intera applicazione.
- Amministrazione più elastica: i processi di sviluppo (build, test, deploy, update) rimangono locali al microservizio.
- Scalabilità efficace: la scalabilità è definita a livello di servizio. In base alle esigenze del sistema, ogni componente può essere replicato e distribuito

sull'hardware che è più adatto in termini di risorse.

Un **servizio upstream** offre dati e/o funzionalità da cui dipendono altri servizi, di solito è la sorgente di flussi di dati.
Un **servizio downstream** utilizza dati e/o funzionalità offerte da altri servizi, di solito è il ricevente di flussi di dati.
La divisione in microservizi upstream e downstream serve a dividere in modo preciso i compiti e le responsabilità di ogni servizio e a stabilire a priori chi si deve gestire e distribuire dati [4].

2.2 Sicurezza nell'utilizzo di microservizi: Zero Trust e SPIFFE

Nella seguente sezione verranno introdotte Zero Trust e SPIFFE, due soluzioni molto diffuse per mettere in sicurezza un sistema che utilizza microservizi, quale una infrastruttura Service Mesh.
Per molti anni le aziende si sono concentrate nel separare e proteggere le loro reti da minacce esterne, con firewall o altri controlli di sicurezza. Assumevano quindi che gli utenti all'interno del perimetro di rete fossero affidabili e quindi potevano consultare tutte le risorse liberamente.
Oggi con la digitalizzazione e la diffusione di ambienti cloud, dispositivi mobili, IoT, SaaS (Software as a Service) e lavoro da remoto, il concetto di perimetro di rete è diventato obsoleto. La superficie di attacco di un'organizzazione si è enormemente ampliata, rendendo inefficaci le difese perimetrali classiche contro minacce interne, violazioni di dati e attacchi come quelli ransomware.
Proposto per la prima volta nel 2010 da John Kindervag (Forrester Research), ***Zero Trust*** è un modello di sicurezza che si basa su un principio cardine: nessun utente, dispositivo o workload è considerato attendibile per impostazione predefinita, neppure all'interno della rete aziendale. Ogni tentativo di accesso deve essere continuamente verificato e autorizzato, applicando controlli rigorosi sulle singole risorse piuttosto che solo sui confini della rete [5].
I tre principi di Zero Trust sono:

- Monitoraggio e convalida continui: utenti, dispositivi e workload devono autenticarsi per accedere a qualsiasi risorsa e ogni volta che richiedono una connessione
- Principio del minimo privilegio: utenti e dispositivi che devono svolgere un'attività o ricoprire un ruolo lo fanno avendo un livello minimo indispensabile di privilegi
- Supposizione di violazione: si parte dal presupposto che le risorse possano già essere compromesse. Di conseguenza, le reti vengono segmentate e ogni attività viene controllata in tempo reale per mitigare e contenere eventuali attacchi

SPIFFE sta per **Secure Production Identity Framework For Everyone** ed è uno standard libero per identificare in modo sicuro i workload nel cloud. Nei moderni ambienti Cloud Native i workload (come Pod o container) vengono generati e distrutti dinamicamente e i nodi vengono aggiunti o rimossi dai cluster dinamicamente, gli indirizzi IP cambiano di continuo. Di conseguenza non è possibile usare indirizzi IP o chiavi statiche per implementare la sicurezza.
SPIFFE propone una soluzione a questo problema fornendo ad ogni workload un'identità crittografica verificabile, il principio è simile ad un certificato SSL di un sito web [6].
Nello specifico, SPIFFE assegna ad ogni workload uno SPIFFE ID, in un formato URI del tipo: spiffe://example.org/ns/default/sa/myserviceaccount .
Nell'URI dato come esempio example.org è il proprio dominio, mentre la parte successiva indica il namespace e il service account (identità digitale dell'applicazione). Il dominio può essere per esempio di un individuo o un'organizzazione che nel proprio sistema utilizza SPIFFE. Ad ogni workload è associato uno SPIFFE Verifiable Identity Document (SVID), firmato dall'autorità del dominio, che fornisce lo SPIFFE ID. Il documento è cifrato e verificabile, così che altri sistemi possono assicurarsi della sua identità. Può essere in formato X.509 oppure in formato JWT Token [6].

X.509 è uno standard introdotto dall'Unione internazionale delle telecomunicazioni (ITU-T) che propone un modello per la validazione dell'identità e la trasmissione di dati criptati. Nello specifico, un certificato X.509 contiene due distinguished name (DN), una firma digitale, il nome dominio del soggetto e la chiave pubblica del soggetto.
Un distinguished name rappresenta l'Autorità di Certificazione e un altro rappresenta l'individuo a cui appartiene il certificato stesso [7].

JSON Web Token (JWT) è uno standard utile per scambiare messaggi tra parti, in modo sicuro, come oggetto JSON. Ogni token è composto da: header, payload, signature. L'header contiene l'algoritmo di cifratura usato. Il payload contiene una dichiarazione (claim) che può essere di tipo registrata, pubblica o privata.
Una dichiarazione registrata serve a trasmettere informazioni che spesso vengono usate dalle applicazioni, il contenuto è standardizzato e quindi non personalizzabile. Una dichiarazione pubblica è pensata per essere usata pubblicamente, può essere per esempio un username. Una dichiarazione privata contiene informazioni sensibili e specifiche dell'organizzazione, può specificare per esempio se un utente è admin [8].

2.3 Orchestrazione

In amministrazione di sistemi, con **orchestrazione** si intende la configurazione automatica, coordinazione, deployment, sviluppo e gestione di computer e software [9].
Essa si distingue dall'**automazione**, che consiste nell'esecuzione di singole attività ripetitive tramite software.
Nei contesti IT moderni, caratterizzati da infrastrutture distribuite tra data center, cloud pubblici e ambienti edge, l'orchestrazione è fondamentale per rendere scalabili i processi automatizzati. Non basta automatizzare i singoli task: è necessario che questi task cooperino in sequenza, comunicando con sistemi esterni e interagendo in modo logico.
I casi d'uso tipici possono essere il Deployment di server o istanze cloud, la risposta automatizzata a incidenti di sicurezza, pipeline DevOps per aggiornamento continuo delle applicazioni [9].

In un contesto cloud, dove sono in esecuzione diversi microservizi, l'orchestrazione è essenziale per fare deployment, gestire, scalare e connettere tra loro container. Con l'orchestrazione di container si può ridimensionare i container in base alle necessità dell'organizzazione e al variare del traffico da parte dei clienti, così da efficientare l'utilizzo di risorse [9].

Il **provisioning** è il processo di allocazione delle risorse e creazione dell'infrastruttura utile per far funzionare uno o più container.

Il **deployment** è la gestione di un'applicazione containerizzata, consiste nel creare il giusto numero di container, aggiornamento a nuove versioni, controllo e ripristino del funzionamento dei container.

Il **bilanciamento del carico** in una piattaforma di orchestrazione consiste nel distribuire il carico tra i container disponibili, al fine di efficientare l'utilizzo di risorse e le prestazioni dell'applicazione. Per fare ciò si può ricorrere a strategie e algoritmi per distribuire al meglio il traffico.

Il **routing del traffico** deve tenere conto del fatto che i container non hanno indirizzi IP statici, in quanto questi possono essere creati e distrutti dinamicamente. Per far si che ogni richiesta raggiunga il corretto container si ricorre quindi a nomi di servizi, ogni container offre uno specifico servizio.

In questo capitolo è stato trattato il concetto di architettura a microservizi ed è stata fornita una definizione di orchestrazione. L'orchestrazione di container è la principale funzionalità di Kubernetes, di cui si parlerà nel capitolo successivo.

Capitolo 3: Kubernetes

Vista la grande diffusione di container per immagazzinare e eseguire applicazioni, ***Kubernetes*** (K8s) nasce per agevolarne la gestione e distribuzione (orchestrazione).
In un ambiente di produzione spesso vengono usati diversi container che devono sempre essere facilmente accessibili e funzionanti (bisogna riavviarli in caso di fallimento). Per fare questo, Kubernetes fornisce un framework robusto per l'esecuzione e la gestione di sistemi distribuiti in modo resiliente. Gestisce automaticamente lo scaling delle applicazioni, il failover in caso di guasti, e supporta diversi modelli di deployment, facilitando così l'implementazione, l'aggiornamento e il mantenimento continuo dei servizi.
Kubernetes si occupa di [9]:

- **Service discovery** e **Load Balancing**: può esporre verso l'esterno un container usando un nome DNS oppure un suo indirizzo IP. Inoltre se il traffico verso un container è alto Kubernetes può agire da Load Balancer, ovvero può distribuire il traffico verso più istanze del container, in modo che il sistema rimanga stabile e non vada in sovraccarico.
- **Storage orchestration**: gestisce automaticamente il mounting (ovvero l'integrazione) di un sistema di storage a scelta dell'utente. Si può configurare Kubernetes in modo che utilizzi risorse di storage locale (su server fisici o virtuali), oppure storage sul cloud (come AWS o Google Cloud).
  In questo modo le applicazioni containerizzate possono accedere ai dati senza dover gestire manualmente la connessione tra container e il sistema di storage.
- Funzionalità di **rollout** (distribuzione) e **rollback** (ripristino) automatizzati per i container: Kubernetes permette di automatizzare il processo di aggiornamento e ripristino dei container in modo sicuro e senza disservizi.
- **Ottimizzazione delle risorse** dei nodi: Kubernetes lavora su un cluster di nodi, si può specificare di quante risorse necessita ogni container (CPU e Ram) così che il sistema possa distribuirli in modo efficiente tra i nodi.
- Gestione della **salute dei container**: riavvia i container che falliscono e li sostituisce.
- Gestione dei **dati sensibili**: può salvare e gestire password, chiavi SSH o simili tramite un sistema chiamato Secrets. Si può aggiornare password o configurazioni delle applicazioni senza dover fermare o ricreare il container.

Nel seguente capitolo verranno illustrate le componenti di Kubernetes e verranno approfondite le funzionalità che offre. Il fine è quello di comprendere il contesto in cui si inserisce una Service Mesh, che si occupa di rendere più efficace l'utilizzo di Kubernetes.

3.1 I componenti di Kubernetes

Un **Pod** è l'unità più piccola e fondamentale di esecuzione in Kubernetes. Contiene uno o più container che condividono lo stesso namespace di rete (IP e porta), lo stesso storage (volumi montati) e possono comunicare tra loro tramite localhost. I Pod vengono usati da Kubernetes per gestire e orchestrare i container, vengono creati e mantenuti attraverso oggetti come i Deployment o i ReplicaSet [9].

Un **Deployment** in Kubernetes è un oggetto che gestisce e controlla il ciclo di vita dei Pod, in particolare quelli che eseguono una determinata versione di un'applicazione. È ciò che l'utente definisce per far girare un'applicazione in modo stabile, replicato e aggiornabile nel tempo [9].

Un **Service** in Kubernetes è un oggetto che fornisce un'interfaccia di rete stabile per accedere a un insieme di Pod. Serve a disaccoppiare la logica applicativa dal dettaglio dell'implementazione, consentendo la scoperta del servizio e il bilanciamento del carico interno al cluster [9].

Un **cluster** Kubernetes è composto da un **Control Plane** e uno o più **nodi worker**. I nodi worker contengono Pod in esecuzione che fanno parte del workload (carico di lavoro). Il Control Plane gestisce i nodi worker e i Pod del cluster. In ambienti di produzione, per garantire alta disponibilità e tolleranza ai guasti, il Control Plane viene eseguito su più nodi master [9].

Un **controller** è un processo che esegue un ciclo infinito per verificare lo stato del cluster. Ha il compito di rendere lo stato del cluster conforme allo stato desiderato, specificato nei file di configurazione delle risorse.
Ogni controller traccia almeno un tipo di risorsa e quando vuole effettuare un cambiamento contatta lo API server [9].

Il progetto ***Container Network Interface*** (CNI), guidato dalla *Cloud Native Computing Foundation* (CNCF), è uno standard e una serie di specifiche e librerie utile per scrivere dei plugin che si occupano di networking tra containers. Questi plugin vengono usati da Kubernetes per configurare le interfacce di rete dei container. Si può scegliere tra diversi CNI purchè siano compatibili con la versione di Kubernetes che si sta usando [9].

I principali componenti del Control Plane sono: kube-apiserver, etcd, kube-scheduler, kube-controller-manager, cloud-controller-manager.

Il ***kube-apiserver*** è la componente front-end del Control Plane, il server che espone l'API di Kubernetes. Tutte le richieste REST verso il cluster Kubernetes vengono gestite attraverso kube-apiserver, che si occupa di validare, autenticare e autorizzare tali richieste, oltre a memorizzare lo stato del cluster nel datastore di backend. Si possono istanziare diverse entità di kubeapiserver per bilanciare il traffico tra loro [9].

Etcd è database chiave-valore distribuito, contiene tutte le configurazioni e lo stato delle risorse Kubernetes. È necessario schedulare un backup di questi dati [9].

Il ***kube-scheduler*** ha il ruolo principale di decidere su quale nodo del cluster posizionare i Pod in base a vari criteri, come le risorse disponibili, le restrizioni di affinità, le priorità di scheduling e altre. Alla creazione di un Pod è il kube-scheduler che decide in che nodo posizionarlo [9].

Il ***kube-controller-manager*** è un componente che esegue i processi controller. Per esempio il Node Controller è responsabile di accorgersi di un eventuale fallimento di un nodo e di riavviarlo [9].

Il ***cloud-controller-manager*** esegue dei controller che interagiscono con il Cloud Provider sottostante. Fornisce le dipendenze necessarie per ogni cloud provider a Kubernetes [9].

Ogni nodo worker deve occuparsi di mantenere i Pod in stato running, sono costituiti dai seguenti componenti: kubelet, kube-proxy, container runtime.

Un ***kubelet*** è un agente in esecuzione in ogni nodo. Si assicura che i Pod e i relativi container siano in esecuzione in modo conforme alle Pod Specifications (PodSpecs). Queste definiscono in maniera dettagliata le caratteristiche e le modalità di esecuzione del Pod stesso [9].

Un ***kube-proxy*** si occupa di networking a livello di nodo, instrada il traffico di rete verso i Pod nel cluster mediante regole iptables e ipvs. Implementa parte del Servizio di Kubernetes [9].

Un **container runtime** è un componente che si occupa di gestire il ciclo di vita dei container nell'ambiente Kubernetes. Tra i container supportati ci sono containerd, CRI-O e Docker. Kubernetes comunica con i container tramite un'interfaccia standard chiamata Container Runtime Interface (CRI) [9].

Uno **Init Container** è un normale container che viene eseguito prima dei container principali di un Pod. Viene eseguito una volta sola e il suo compito deve

terminare con successo. Si occupa di predisporre l'ambiente di lavoro del Pod, quindi per esempio aspetta che alcuni servizi siano pronti e copia file di configurazione [9].

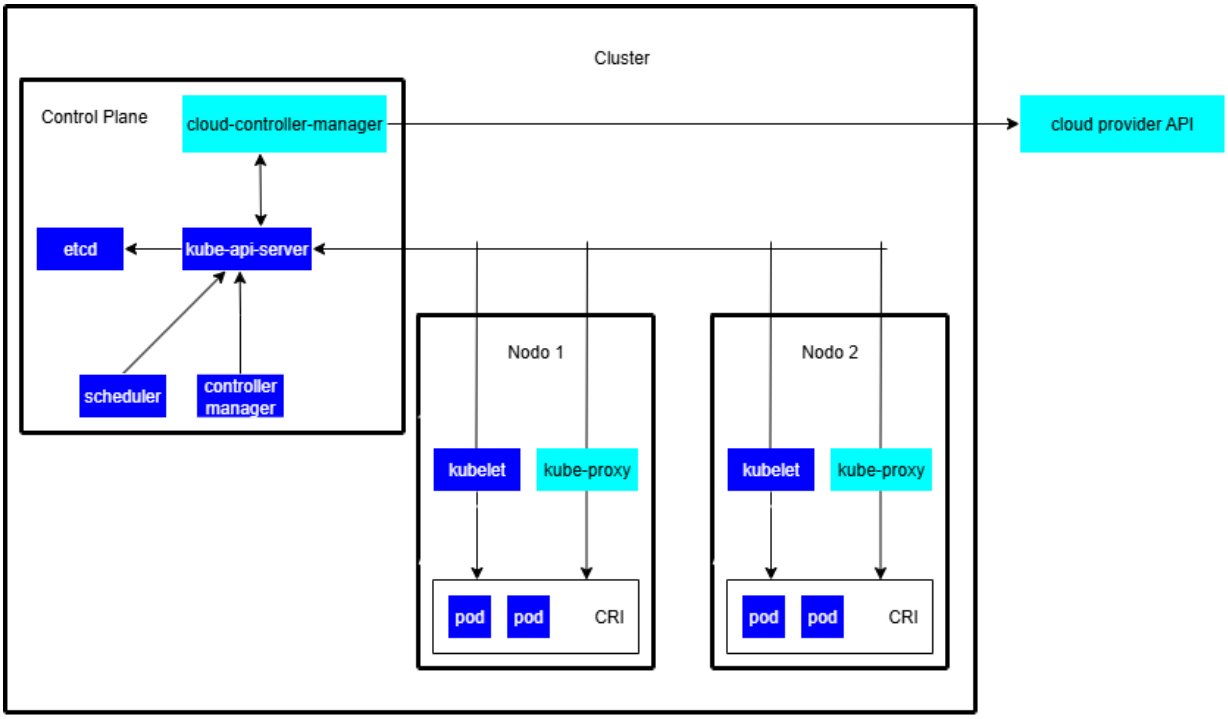

Figura 3.1: Componenti di un cluster Kubernetes

In Figura 3.1 si può vedere uno schema di funzionamento delle componenti di Kubernetes. All'interno di un cluster i nodi riferiscono tutti a un nodo master su cui risiede il Control Plane. In ogni nodo si può vedere le sue principali componenti come il kubelet, il kube-proxy e i Pod.

Kubernetes fornisce uno strumento chiamato ***kubectl*** per interagire con il Control Plane del cluster, usando le API Kubernetes. È utile per esempio per visualizzare le risorse del cluster, applicare file di configurazione, aprire una shell dentro ad un container, visualizzare i log di un container.

3.2 Addon

Gli addon sono componenti opzionali del cluster che arricchiscono le sue funzionalità. Sono tipicamente implementati come Pod, controller o Service e operano interagendo con il Control Plane. Alcuni di questi sono [9]:

- ***Cluster DNS***: responsabile del servizio di risoluzione DNS interno al cluster, fondamentale per la comunicazione tra Pod e servizi. I container si affidano in automatico a lui per le loro risoluzioni.
- **Web UI** (Dashboard): UI web-based per il general purpose, permette agli utenti di gestire e verificare le applicazioni in esecuzione nel cluster, ma

anche il cluster stesso.

- ***Container Resource Modelling***: registra dati generici a intervalli regolari sui container in un database centrale, offre una UI per visionare i dati.
- **Cluster-level Logging**: responsabile per il salvataggio dei log dei container in un volume centralizzato. Offre un'interfaccia di visualizzazione Web.

3.3 Oggetti in Kubernetes

Con **oggetto** in Kubernetes si intende un'entità persistente nel cluster che rappresenta lo stato desiderato di una risorsa del cluster. Gli oggetti costituiscono la principale modalità con cui l'utente dichiara e configura ciò che Kubernetes deve creare, gestire e mantenere nel tempo. Ogni oggetto Kubernetes è una risorsa dichiarativa, definita tramite un file ***YAML*** o ***JSON***, che specifica in modo esplicito: il tipo di oggetto (es. Pod, Service, Deployment, ConfigMap, ecc.), i metadati (nome, namespace, etichette, annotazioni) e lo stato desiderato (es. quali container eseguire, quante repliche mantenere, policy di rete o di sicurezza) [9].

Il cluster Kubernetes, attraverso i controller, ha quindi il compito di creare e mantenere nello stato desiderato gli oggetti. Pod, Deployment e Service, introdotti precedentemente, sono tra gli oggetti più comuni. Di seguito verranno introdotti altri esempi di oggetti.

Un **ReplicaSet** è un oggetto che ha il compito di far sì che ci sia un preciso numero di replica Pod in esecuzione. Crea i Pod (usando un template) e ne distrugge se necessario. Questo tipo di oggetto non viene spesso usato da solo ma viene implementato dagli oggetti Deployment per integrare le loro funzionalità [9].

Un **StatefulSet** è uno oggetto usato dalle applicazioni che richiedono che i Pod abbiano un nome DNS e uno storage persistente, per esempio applicazioni di database [9].

Un **DaemonSet** è un oggetto che mette la copia di un Pod in esecuzione su un gruppo prestabilito di nodi. È utilizzato, per esempio, per avere un Pod che si occupa dei Log in ogni nodo [9].

Un **ConfigMap** è un oggetto utilizzato per separare la configurazione dall'immagine applicativa. Serve a conservare dati non sensibili sotto forma di coppie chiave-valore. Questi dati possono essere per esempio variabili d'ambiente, argomenti da passare a riga di comando, file di configurazione montati in un volume [9].

Un **Secret** è un oggetto che conserva alcuni dati sensibili come password o token. Questi dati possono essere, per esempio, chiavi SSH da fornire ad un Pod. Usando un Secret si può evitare di usare dati sensibili all'interno del codice dell'applicazione [9].

Un **Ingress** è un oggetto che espone servizi del cluster verso l'esterno, che saranno raggiungibili tramite richieste HTTP e HTTPS [9].

Il file di configurazione YAML, detto manifest, deve contenere:

- **apiVersion**: la versione dell'API Kubernetes da utilizzare
- **kind**: il tipo di oggetto da creare (es. Pod, Deployment)
- **metadata**: identificatori dell'oggetto, come name, labels, namespace
- **spec**: lo stato desiderato dell'oggetto (contenuti, repliche, policy, ecc.)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  replicas: 2
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

Figura 3.2: Esempio di file .yaml

Il file YAML sopra riportato definisce un oggetto di tipo Deployment. "replicas: 2" significa che verranno creati due Pod identici. In questo caso, visto che non è specificato il registry dell'immagine, questa verrà presa dal registro pubblico Docker Hub all'indirizzo docker.io/library/nginx:1.14.2.

Per applicare un manifest definito in un file YAML si può usare il comando: "kubectl apply –f nome_manifest.yaml".

3.4 Custom Resources

Kubernetes salva i suoi oggetti standard (come Pod, Deployment ecc.) in collezioni, all'interno del suo database. Si può interagire con queste risorse con la Kubernetes API.
Spesso nei progetti che utilizzano Kubernetes sorge la necessità di aggiungere

nuovi oggetti ed estendere le API Kubernetes. Per farlo Kubernetes propone le **Custom Resources**, che servono a estendere le funzionalità del cluster in modo coerente al modello dichiarativo dei file di configurazione YAML e compatibile con il command line tool kubectl [9].

Una **Custom Resource Definition** (CRD) è un file di configurazione YAML che serve a dichiarare una nuova risorsa. Specifica nome e gruppo API della nuova risorsa, versione, scopo e altre informazioni che serviranno al controller per gestire tale risorsa.

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: databases.miodominio.com
spec:
  group: miodominio.com
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                engine:
                  type: string
                version:
                  type: string
  scope: Namespaced
  names:
    plural: databases
    singular: database
    kind: Database
```

Figura 3.3: Esempio di CRD

In Figura 3.3 viene mostrato un esempio di CRD. Il campo “group” serve a specificare il gruppo API della risorsa, così da non confonderla con le API standard. Nel campo “versions” viene definita una versione “v1” esposta da Kubernetes tramite le API. La risorsa accetta due campi chiamati “engine” e “version”, di tipo stringa. Nel campo “names” vengono specificati i nomi con cui Kubernetes identificherà la risorsa.

```yaml
apiVersion: mycompany.com/v1
kind: Database
metadata:
  name: mydb
spec:
  engine: postgres
  version: "14"
```

Figura 3.4: Esempio definizione di un oggetto di un tipo definito con una CRD

In Figura 3.4 viene mostrato un esempio di definizione di un oggetto “Database” che si basa sulla risorsa definita nel CRD di Figura 3.3.

Se si definisce una nuova Custom Resource serve un controller che dia vita alla sua logica. Questo può essere creato, per esempio, con la libreria ***client-go*** di Go per interagire con l'API Server, oppure con il framework ***Kubebuilder***.

Continuando con l'esempio, la risorsa creata in Figura 3.4 deve essere gestita da un controller, che può essere quello fornito dal database stesso oppure uno scritto apposta.

3.5 Kubernetes Self-Healing

Kubernetes si occupa di mantenere il sistema nello stato desiderato, ovvero quello specificato dai file di configurazione applicati dagli amministratori di rete. Deve quindi assicurarsi che ogni workload sia disponibile e funzionante, se avviene qualche problema deve risolverlo e ripristinarne il corretto funzionamento. Quindi, per esempio, deve riavviare i container che falliscono, assicurarsi che ci sia il corretto numero di Pod in funzione (secondo le istruzioni di ReplicaSet, StatefulSet e DaemonSet), inoltrare le richieste per un servizio solo ai Pod funzionanti [9].

Tra i compiti dei kubelet di ogni nodo c'è quello di mantenere in salute i container, seguendo le istruzioni dettate da dei meccanismi di health-check chiamati Liveness Probe, Readiness Probe e Startup Probe. Questi vengono specificati nei file di configurazione di Pod, Deployment ecc.

Una **Liveness Probe** ha lo scopo di verificare se un container è ancora funzionante e specifica quando va riavviato. Serve a identificare deadlock, cicli infiniti o arresti anomali. L'operazione di verifica consiste nel chiedere al container il suo stato di salute, lo si può fare attraverso un comando (apertura di un file del container che contiene l'informazione), una richiesta HTTP o gRPC oppure una richiesta di apertura di una socket TCP [9].

```
1 livenessProbe:
2   httpGet:
3     path: /healthz
4     port: 8080
5   initialDelaySeconds: 5
6   periodSeconds: 10
```

Figura 3.5: Esempio di Liveness Probe

In Figura 3.5 viene riportato un esempio di Liveness Probe, in cui viene specificato di interrogare l'endpoint HTTP dell'applicazione che si occupa di riferire lo stato di salute della stessa.

Una **Readiness Probe** serve a verificare se un container è pronto a ricevere

traffico. Solo i Pod in stato Ready posso ricevere richieste per un servizio e gli viene dato del tempo per avviarsi correttamente e quindi cambiare stato. Come nel caso della Liveness Probe, l'operazione di verifica viene fatta mediante un comando, una richiesta HTTP o gRPC, oppure una richiesta di apertura di una socket TCP [9].

```
1 readinessProbe:
2       tcpSocket:
3         port: 8080
4       initialDelaySeconds: 15
5       periodSeconds: 10
```

Figura 3.6: Esempio di Readiness Probe

Una **Startup Probe** serve a tutelare i container che richiedono molto tempo per avviarsi e funzionare correttamente. A questi container non vengono quindi applicate le regole di Liveness Probe e Readiness Probe fino a quando il container non si è avviato con successo [9].

```
1 startupProbe:
2   httpGet:
3     path: /healthz
4     port: 8080
5   failureThreshold: 30
6   periodSeconds: 10
```

Figura 3.7: Esempio di Startup Probe

In questo capitolo sono state illustrate le principali funzionalità di Kubernetes, nel prossimo capitolo verrà spiegato il concetto di Service Mesh, al fine di comprendere quali problemi può risolvere in un cluster.

Capitolo 4: Service Mesh

Con l'aumento del numero di microservizi coinvolti in un'applicazione cresce esponenzialmente la complessità del sistema. Sorgono quindi problemi legati al routing del traffico, alla sicurezza, al monitoraggio delle chiamate ai servizi, al bilanciamento del carico e alla gestione degli errori. Inizialmente, tali problematiche venivano gestite direttamente all'interno del codice di ciascun microservizio, con soluzioni personalizzate che però portavano a una duplicazione delle logiche di comunicazione, rendendo il sistema fragile e difficile da manutenere [10].

Si è poi sviluppato il concetto di ***Service Mesh***, una soluzione infrastrutturale che si inserisce tra i microservizi, con il compito di gestire in maniera trasparente tutte le comunicazioni tra essi. Una Service Mesh non aggiunge nuove funzionalità all'applicazione in sé, ma ne migliora l'interoperabilità e la stabilità, fornendo un livello di astrazione che centralizza la gestione della comunicazione service-to-service [10].

Una Service Mesh si basa su due componenti fondamentali: il Data Plane e il Control Plane. Il **Data Plane** è costituito da una serie di proxy distribuiti, detti **sidecar proxy**, che vengono affiancati a ogni microservizio (di solito all'interno dello stesso Pod nei container orchestrati, come in Kubernetes). Questi proxy intercettano e gestiscono tutto il traffico in entrata e in uscita dai servizi, occupandosi di funzioni come bilanciamento del carico, retry automatici, gestione dei timeout, crittografia delle comunicazioni, controllo degli accessi e raccolta di metriche. In questo modo, la logica di comunicazione viene spostata dai microservizi stessi alla rete di proxy, alleggerendo il codice e uniformando il comportamento del sistema [10].

Il **Control Plane**, invece, è responsabile della configurazione dei proxy, della distribuzione delle politiche di sicurezza e delle regole di routing, e della raccolta di dati di telemetria. Ha il compito di definire e propagare le impostazioni che regolano il comportamento della Service Mesh, facilitando il lavoro dell'amministratore di sistema che deve gestire il sistema e risolvere eventuali problemi. Attraverso il Control Plane, è possibile modificare in tempo reale il comportamento della rete, ad esempio per deviare il traffico durante un aggiornamento, implementare una nuova versione di un servizio o reagire a situazioni anomale [10].

Un aspetto chiave della Service Mesh è che essa funziona senza modificare il codice applicativo. Tutti i vantaggi offerti (come l'osservabilità, la sicurezza end-to-end, la resilienza e l'affidabilità) sono ottenuti senza che gli sviluppatori debbano integrare manualmente tali funzionalità nei propri microservizi. Ciò

consente di separare nettamente le preoccupazioni infrastrutturali da quelle di business, permettendo ai team di concentrarsi interamente sulla logica dell'applicazione [10].

L'adozione di una Service Mesh è quindi particolarmente indicata in contesti in cui la comunicazione tra microservizi è intensiva, come nei grandi sistemi e-commerce, nei servizi cloud, nei social network e in tutte quelle applicazioni che richiedono alta disponibilità, scalabilità dinamica e tracciabilità dei flussi di traffico.
Ad oggi esistono diversi sistemi di Service Mesh che possono essere configurati per integrarsi al meglio nei sistemi di gestione di container esistenti.

In sintesi, il concetto di Service Mesh rappresenta una risposta architetturale avanzata ai limiti emergenti delle architetture a microservizi, garantendo un'infrastruttura di comunicazione efficiente, sicura e osservabile. La sua adozione consente alle organizzazioni di costruire applicazioni moderne più robuste, semplificando al contempo la gestione operativa e accelerando lo sviluppo.

Kubernetes fornisce strumenti per fare Service Discovery e Load Balancing (attraverso gli oggetti Service e Ingress), ma non offre funzionalità come il controllo del traffico e l'osservabilità distribuita. Per fare ciò si possono utilizzare le tecnologie di Service Mesh, che integrano perfettamente con l'architettura a Pod di Kubernetes utilizzando il pattern sidecar proxy. Questi ultimi vengono affiancati ad ogni Pod per intercettare e gestire il loro traffico.

Nel seguente capitolo verrà introdotto uno strumento chiamato Envoy che è spesso utilizzato da sistemi di Service Mesh. Infine, verrà spiegato meglio l'obiettivo di questo lavoro.

4.1 Envoy

Envoy è spesso usato come Data Plane in una Service Mesh. Si tratta di un service-to-service communication proxy sviluppato da Lyft, ovvero un proxy tra applicazioni che si occupa di Load Balancing, supporto per protocolli HTTP/2, gRPC, e TLS, osservabilità nativa (metrics, logging e tracing), hot reload delle configurazioni e resilienza tramite circuit breaker e retry. Envoy inoltra e gestisce il traffico tra servizi in ogni singolo nodo, il sistema di Service Mesh invece controlla e coordina il comportamento di tanti proxy Envoy all'interno del cluster [11].

In particolare, Envoy si occupa di [11]:

- **Service discovery** e **configurazione dinamica**

- **Health Checking**: Envoy invia richieste periodiche ai servizi per verificarne lo stato e rileva automaticamente i servizi malfunzionanti analizzando gli errori nelle risposte. Il bilanciamento del carico viene quindi fatto anche con questi dati raccolti
- **Bilanciamento del carico** avanzato: Envoy implementa retry automatici (ritenta la richiesta di un servizio se c'è un fallimento), circuit breaking (vieta il traffico verso un Pod che è in sovraccarico) e global rate limiting (limitare il numero di richieste che possono essere inviate a un servizio entro un determinato periodo di tempo)
- **Osservabilità**: a livello di rete e applicativo

Tra i sistemi di Service Mesh che utilizzano Envoy Proxy vi sono: Istio, AWS App Mesh, Kuma.

4.2 Tecnologie per Service Mesh: Istio e Network Service Mesh

Obiettivo di questa tesi sarà confrontare e valutare le funzionalità e le prestazioni di due famose tecnologie di Service Mesh: Istio e Network Service Mesh.

Istio è un progetto ideato dalla collaborazione tra Google, IBM e Lyft, annunciato nel 2017. Si tratta di una classica Service Mesh, che propone soluzioni per gestire e far comunicare in modo efficiente microservizi, offrendo funzionalità di sicurezza, monitoraggio e gestione del traffico e aggiornamenti. Oggi il progetto Istio è mantenuto dalla community della Cloud Native Computing Foundation (CNCF).

Network Service Mesh (NSM) è un progetto che fa parte della Cloud Native Computing Foundation (CNCF) dal 2019. Non si tratta di un classico Service Mesh, anche se offre funzionalità simili. Non si concentra solo sulla comunicazione tra microservizi, ma anche sulla connessione di servizi tra reti diverse. Infatti NSM offre funzionalità per collegare servizi di rete tra cluster, data center e sistemi on-premise. Il progetto vuole rendere le reti più flessibili, sicure e facili da gestire, permettendo alle applicazioni di comunicare senza doversi preoccupare di come sia configurata la rete fisica o virtuale.

Nei capitoli successivi verranno introdotte le due tecnologie, al fine di comprendere la loro architettura, il modo di funzionamento e le funzionalità che offrono quali l'osservabilità, la gestione del traffico, la sicurezza, l'astrazione dei servizi di rete. Verrà posta particolare attenzione a quali sono le similarità tra le due tecnologie e come, invece, lavorano in modo diverso.
Infine, verranno illustrati i test che sono stati svolti con le due tecnologie. Questi

hanno avuto lo scopo di valutare le prestazioni delle due tecnologie in utilizzi diversi, insieme al consumo di risorse del cluster che queste hanno comportato.

Capitolo 5: Istio

Kubernetes non gestisce aspetti come il controllo del flusso di traffico in entrata e in uscita tra i microservizi, l'abilitazione di regole di routing avanzate per un controllo granulare del traffico, o la raccolta di dati di telemetria di rete dai singoli microservizi. Per fare queste cose è necessario uno strumento separato.
Istio è una piattaforma open-source per la gestione del traffico di rete tra i microservizi. Fornisce un Service Mesh, ovvero un livello di infrastruttura che consente di controllare in modo dettagliato il comportamento del traffico, senza richiedere modifiche al codice dell'applicazione. Istio si basa su Envoy, che viene affiancato a ciascun microservizio per intercettare e gestire il traffico in ingresso e in uscita.

In particolare, Istio si occupa di [12]:

- Implementare **connessioni sicure** tra microservizi all'interno di un cluster tramite mTLS, con procedure di autenticazione e autorizzazione
- **Bilanciamento del carico** automatico per traffico HTTP, gRPC, WebSocket (L7) e TCP (L4)
- Controllo dettagliato del comportamento del traffico grazie a **regole di routing** avanzate, **retry** automatici, **failover** e iniezione di errori (**fault injection**).
- Istio permette di definire, modificare ed estendere le regole di accesso e controllo del traffico in modo modulare, cioè tramite componenti che si possono "inserire" o "scollegare". Fornisce un'**API di configurazione** che supportano controlli di accesso e limiti di velocità (rate limiting).
- **Metriche**, **log** e **tracing** generati automaticamente per tutto il traffico all'interno del cluster, inclusi traffico in ingresso (ingress) e in uscita (egress).

Istio può essere utilizzato anche in sistemi locali (on-premesis). Istio si integra perfettamente con il CNI del cluster.

Nel seguente capitolo verrà illustrata l'architettura di Istio e le sue principali funzionalità. Verrà posta particolare attenzione per le funzionalità che Istio ha in comune con NSM, come osservabilità, sicurezza, controllo della salute del sistema e integrazione con sistemi legacy.

5.1 Architettura

I componenti principali di Istio sono il Data Plane e il Control Plane.
Il **Data Plane** contiene il proxy Envoy e si occupa di [12]:

- **Service discovery**: per consentire ai microservizi di identificarsi a vicenda nella rete
- **Load balancing**: per distribuire le richieste provenienti dalla rete in modo efficiente tra i microservizi
- **TLS termination**: decifra il traffico della rete per i microservizi
- **Health checks**: per monitorare la disponibilità dei microservizi e assicurarsi che siano raggiungibili nella rete
- **Raccolta dati**: riguardo la salute e la reattività di un microservizio

Il deployment dei proxy Envoy avviene mediante **modello sidecar**, ovvero un container aggiuntivo che viene eseguito accanto a un'applicazione principale all'interno dello stesso Pod. Envoy intercetta il traffico in entrata e in uscita dall'applicazione al fine di svolgere i compiti a lui assegnati e raccogliere dati [12].

Il **Control Plane** di Istio si chiama ***istiod*** e si occupa di determinare le policy di rete e di sicurezza. Gestisce e configura i proxy Envoy. In particolare si occupa di [12]:

- **Gestione del ciclo di vita dei proxy**: questa funzionalità è gestita da una parte di Istiod chiamata Pilot. Quest'ultimo monitora le regole di routing di alto livello e le policy di sicurezza configurate all'interno di Istio, e le traduce in istruzioni per ciascun proxy sidecar.
- Gestione della **sicurezza**: utilizzando un sotto-componente chiamato Citadel, Istiod si occupa della gestione delle policy di sicurezza per i proxy Envoy. Più precisamente, Citadel gestisce attività come l'autenticazione degli utenti, la gestione dei certificati e l'assicurazione della cifratura dei dati di rete.
- **Isolamento dell'infrastruttura**: Mixer è il componente del Control Plane che isola le applicazioni dall'infrastruttura backend. Lo fa operando come intermediario tra le applicazioni e l'infrastruttura che le ospita, controllandone il comportamento in conformità con le policy configurate in Istio. Mixer è anche responsabile della raccolta dei dati di telemetria associati alle chiamate di servizio e consente l'applicazione dei controlli di accesso.

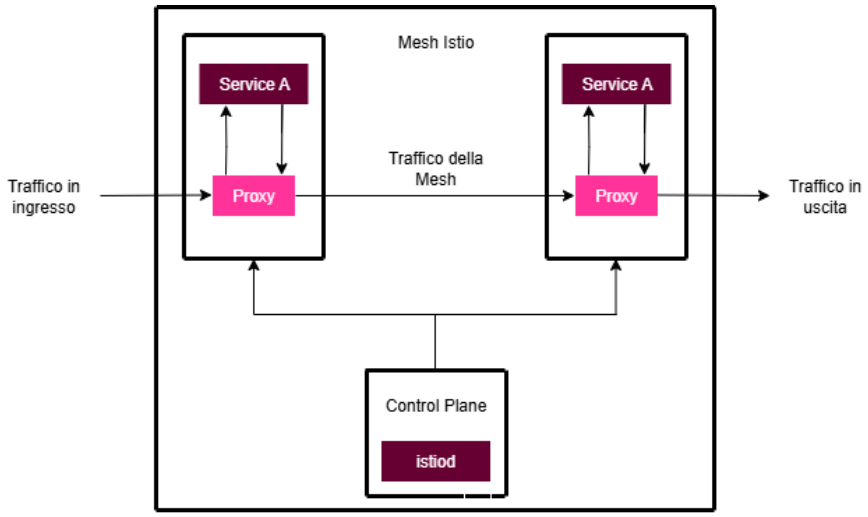

Figura 5.1: Architettura di Istio

In Figura 5.1 si può vedere uno schema di funzionamento di una Service Mesh Istio in cui il Control Plane dirige il funzionamento dei sidecar proxy. Il traffico tra servizi e con l'esterno viene intercettato dai proxy.

5.2 Esempi di utilizzo di Istio

In una Mesh Istio a singolo cluster il funzionamento di Istio è piuttosto semplice. Tutti i componenti del Data Plane e del Control Plane operano all'interno dello stesso cluster Kubernetes. In questo caso, probabilmente la latenza di rete sarà minima e le attività come il bilanciamento del carico e la scoperta dei servizi saranno relativamente semplici, poiché tutto funziona all'interno dello stesso cluster [12].

In una Mesh multi-cluster, una singola Mesh di servizi Istio connette i servizi in esecuzione su più cluster Kubernetes. È possibile implementarla distribuendo un unico Control Plane all'interno di un cluster e configurandolo per gestire i proxy Envoy in esecuzione su tutti i cluster che si desidera controllare.
In alternativa, si può distribuire un'istanza diversa del Control Plane in ciascun cluster, che gestisce solo i proxy locali. In questo modo il sistema è più resiliente al fallimento di un cluster o del suo Control Plane ma richiederà più risorse e sarà meno efficiente [12].

Una Mesh multi-cluster può essere soggetta a:

- elevata latenza di rete se i cluster si trovano molto distanti geograficamente
- difficoltà nel Service Discovery se per esempio un servizio la subnet in cui risiede un servizio è protetta da un firewall
- maggiore overhead: un numero maggiore di Control Plane comporta un overhead più elevato.

5.3 Gestione del traffico

Istio permette di configurare delle regole di instradamento in modo da controllare il flusso del traffico e le chiamate API tra i vari servizi. Per esempio, si possono configurare proprietà a livello di servizio come circuit breaker, timeout e tentativi di ritrasmissione. Si possono anche fare dei test A/B tra servizi e rollout canary, per confrontare una nuova versione di un servizio con quella vecchia, in tempo reale. Ci sono anche meccanismi per migliorare la capacità di sopravvivenza di un'applicazione a microservizi a fronte di malfunzionamenti della rete o di servizi da cui dipende [12].
La gestione del traffico di Istio è affidata ai proxy Envoy che sono affiancati ad ogni servizio. Tutto il traffico di una Mesh passa da questi ultimi, quindi lo si può gestire senza apportare alcuna modifica ai servizi.

Per dirigere il traffico della Mesh Istio deve conoscere indirizzo IP e porta di ogni endpoint (di ogni Pod) e quale servizio offrono. Queste informazioni vengono conservate in un Service Registry e servono per generare le configurazioni di Envoy. Istio non implementa un suo sistema di Service Discovery ma utilizza quello preesistente dell'infrastruttura su cui opera. Per popolare il suo registry, per esempio, utilizza le API di Kubernetes per interrogare il Registry del cluster [12].

Partendo da una semplice Service Mesh fatta con Istio che include Service Discovery e Load Balancing si può introdurre tante personalizzazioni per controllare il traffico, per esempio [12]:

- **A/B testing**
- applicare una differente **policy di Load Balancing** per un gruppo di servizi (che magari risiedono in nodi più o meno performanti)
- **Controllare il traffico** che entra nella Mesh (ingress traffic) provenienti da client esterni, oppure controllare il traffico che esce dalla Mesh (egress traffic) per raggiungere servizi esterni e/o API.
  Si possono intensificare le politiche di sicurezza (autenticazione per esempio) oppure gestire i time-out e i retries

Per implementare le sue funzionalità Istio utilizza le Custom Resource di Kubernetes. Istio definisce diverse Custom Resource Definition (CRD) per gestire il traffico: Virtual Services, Destination Rules, Gateways, Service Entries e Sidecars.

5.4 Virtual Services

Envoy di default fa Load Balancing con una strategia chiamata Least Request Load Balancing, ovvero inoltra ogni richiesta all'istanza del servizio che ne sta già servendo di meno. Un Virtual Service di Istio è una risorsa che consente di configurare come le richieste vengono instradate ad un servizio all'interno della Mesh attraverso una serie di regole. Quando arriva una richiesta Istio scorre ogni regola in ordine e quando ne trova una che fa match (con la richiesta stessa) la inoltra al corrispondente Virtual Service. In generale si può dirottare il traffico per un Virtual Service a diverse destinazioni. Per esempio si può fare A/B testing: dirottare parte del traffico ad una nuova versione di un servizio in proporzioni prestabilite, così da verificarne il funzionamento. Si può anche fare un canary rollout e quindi aumentare progressivamente la quantità di traffico che si manda ad un Servizio, per vedere come reagisce.
Infine i Virtual Service vengono spesso usati per indirizzare diverse applicazioni ad uno stesso Virtual Service, in questo modo si può per esempio convertire un'applicazione monolitica ad una a microservizi senza richiedere al cliente di adattarsi alla nuova infrastruttura [12].

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: canary-deployment
spec:
  hosts:
  - my-service
  http:
  - route:
    - destination:
        host: my-service
        subset: v1
      weight: 90
    - destination:
        host: my-service
        subset: v2
      weight: 10
    retries:
      attempts: 3
      perTryTimeout: 2s
    fault:
      delay:
        fixedDelay: 5s
        percent: 50
      abort:
        httpStatus: 500
        percent: 50
```

Figura 5.2: Esempio di Virtual Service

Quello mostrato in Figura 5.2 è un Virtual Service nominato "canary-deployment". Il 90% del traffico verrà dirottato alla versione v1 del deployment nominato "my-service", il restante verrà dirottato alla versione v2 dello stesso deployment. Se una richiesta fallisce Envoy la ritenterà tre volte a distanza di due

secondi ogni volta. Viene anche stabilita una Fault Injection che si può usare, in questo caso, per verificare come si comporta il servizio quando il 50% delle richieste vengono abortite (HTTP 500 error) e l'altro 50% presa a carico con un ritardo di cinque secondi.

5.5 Destination Rules

Una Destination Rule serve a determinare cosa accade al traffico che un Virtual Service ha dirottato verso un determinato servizio. In una Destination Rule viene definito un sottoinsieme (subset) di servizi che si contraddistinguono per una label. Si può poi definire delle regole di instradamento per ogni istanza del servizio [12].

Per esempio, nel subset si può definire due versioni diverse v1 e v2 dello stesso servizio e poi indicare un modello da usare per inoltrare il traffico ai subset.
Envoy utilizza il modello Least Requests, mentre con una Destination Rule si possono usare i seguenti modelli [12]:

- **Random**: le richieste vengono inoltrate a caso
- **Weighted**: le richieste vengono inoltrate alle istanze del servizio secondo proporzioni precise
- **Round Robin**: le richieste vengono inoltrate ad ogni istanza in sequenza
- **Consistent Hash**: cerca di inoltrare le richieste di uno stesso client sempre alla stessa istanza del servizio. Per fare ciò si basa sugli Header HTTP, cookies e IP sorgente
- **Ring Hash**: consiste nel mappare le richieste (basandosi su Header HTTP e cookies) a specifiche istanze di un servizio, anche quando il numero di istanze cambia.

```yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: my-destination-rule
spec:
  host: my-svc
  trafficPolicy:
    loadBalancer:
      simple: RANDOM
  subsets:
  - name: v1
    labels:
      version: v1
  - name: v2
    labels:
      version: v2
    trafficPolicy:
      loadBalancer:
        simple: ROUND_ROBIN
  - name: v3
    labels:
      version: v3
```

Figura 5.3: Esempio di DestinationRule

In Figura 5.3 viene riportato un esempio di Destination Rule in cui ogni subset è identificato da una label. Queste verranno messe da Kubernetes nel deployment del servizio come metadata, per identificare le differenti versioni. Ad ogni Pod sarà associata una label.
Viene poi definita una politica per il traffico di tipo RANDOM, che viene applicata a tutti i subset a parte quello nominato v2 a cui viene assegnato un modello ROUND_ROBIN.

5.6 Gateways

Per gestire il traffico in entrata e uscita dal cluster Kubernetes propone uno strumento chiamato Ingress che si occupa di esporre all'esterno delle rotte HTTP e HTTPS. Il traffico viene gestito da un Ingress Controller (per esempio NGINX). Ingress espone servizi di tipo NodePort oppure LoadBalancer. I servizi NodePort sono raggiungibili all'IP di un nodo del cluster ad una porta specifica, mentre i servizi LoadBalancer sono raggiungibili ad un IP fornito da un Cloud Provider. Il Kubernetes Ingress non consente di usare mTLS e controllare il traffico; è dipendente dall'Ingress Controller [9]

Un Istio Gateway è una risorsa Kubernetes che configura un proxy Envoy per gestire il traffico in ingresso alla Mesh. Non prevede Ingress Controller e permette una separazione delle configurazioni di livello 4 e 7. Con un Gateway si può definire le porte e i protocolli in ascolto, i certificati TLS, le interfacce di rete. Per gestire il traffico HTTP viene affiancato ad un VirtualService. Quindi, permette di gestire il traffico in entrata/uscita dalla Mesh allo stesso modo di ogni altro Data Plane (traffico service-to-service). Si può quindi usare le funzionalità

di Istio allo stesso modo, usando Virtual Service e Destination Rule [12].

Con una risorsa Kubernetes di tipo Gateway si può anche configurare un Egress Gateway, con cui gestire il traffico in uscita. Si può stabilire quali servizi possono avere un accesso sicuro alla rete e tramite quale nodo. In questo modo servizi esterni vengono chiamati direttamente dal sidecar del client.
Un Egress Gateway può essere utile per esempio nel caso in cui un organizzazione sia vincolata da una policy di sicurezza a far uscire il traffico per la rete solo da alcuni nodi, oppure nel caso in cui i nodi su cui risiedono le applicazioni non possiedono IP pubblici. Quindi si procede conferendo un IP pubblico all'Egress Gateway e il traffico passerà da lui [12].

Istio fornisce due deployment preconfigurati di Gateway Proxy chiamati istio-ingressgateway e istio-egressgateway [12].

```yaml
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: my-gateway
  namespace: default
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 80
      name: http
      protocol: HTTP
    hosts:
    - "myapp.example.com"
```

Figura 5.4: Esempio di utilizzo di istio-ingressgateway

In Figura 5.4 viene riportato un esempio di file di configurazione YAML di un Gateway che ascolta traffico HTTP alla porta 80 del dominio myapp.example.com. Ci sarà anche bisogno di un Virtual Service che indirizza le richieste provenienti dal gateway ad un servizio, per esempio:

```yaml
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: myapp-vs
  namespace: default
spec:
  hosts:
  - "myapp.example.com"
  gateways:
  - my-gateway
  http:
  - match:
    - uri:
        prefix: /
    route:
    - destination:
        host: myapp-service
        port:
          number: 80
```

Figura 5.5: VirtualService

5.7 Service Entries

Ogni risorsa Kubernetes definita come "service" all'interno di una Mesh viene in automatico scoperta da Istio (Service Discovery), consentendo ai servizi di comunicare tra loro. I servizi esterni al cluster non vengo scoperti in modo automatico, ma serve una Service Entry per aggiungere una entry al Service Registry di Istio. In questo modo i proxy Envoy possono mandare traffico al servizio come se appartenesse alla Mesh locale. Configurare una Service Entry consente di gestire il traffico di servizi che sono in esecuzione fuori dalla Mesh, quindi per esempio ridirigere e inoltrare il traffico a destinazioni esterne, definire retry e timeout oppure eseguire un servizio che risiede in una macchina virtuale come se fosse nella Mesh [12].

```yaml
apiVersion: networking.istio.io/v1
kind: ServiceEntry
metadata:
  name: svc-entry
spec:
  hosts:
  - ext-svc.example.com
  ports:
  - number: 443
    name: https
    protocol: HTTPS
  location: MESH_EXTERNAL
  resolution: DNS
```

Figura 5.6: Esempio di Service Entry

Con il file di configurazione di Figura 5.6 viene definita una Service Entry che consente ai servizi all'interno della Mesh di raggiungere https://ext-svc.example.com .

5.8 Osservabilità

Istio ascolta tutte le comunicazioni tra i servizi della Mesh e genera dati telemetrici. Il comportamento dei servizi viene quindi osservato da Istio e fornito a chi gestisce l'infrastruttura, aiutandolo a risolvere problemi e ottimizzarne il lavoro. Istio raccoglie dati su come i servizi interagiscono tra loro e con i componenti di Istio stesso, attraverso metriche, tracce distribuite e log di accesso [12].

Istio genera delle **metriche** per tutto il traffico entrante e uscente dalla Mesh e il traffico interno. Vengono raccolti dati come il volume di traffico, il tasso di errore e il tempo di risposta. Sono anche raccolti dati sul funzionamento del Control Plane della Mesh [12].

Ogni sidecar proxy Envoy raccoglie metriche riguardo al traffico che gestisce e alle funzioni che esso deve svolgere [12].
In particolare, Envoy può essere configurato per raccogliere più o meno dati statistici, a seconda della quantità di risorse CPU che è possibile dedicargli. Le statistiche possono essere di tre tipi: downstream (richieste di connessione), upstream (comunicazioni in corso) e server (come Envoy lavora, per esempio uptime o utilizzo della memoria) [11].
Istio raccoglie metriche anche a livello di servizio, quali latenza, traffico, errori, e saturazione.

Le metriche standard di Istio vengono esportate in automatico su **Prometheus**, un sistema di monitoraggio e database di serie temporali. Prometheus estrae le metriche esposte da componenti Kubernetes e da Istio (da Envoy proxy e istiod) e le salva come serie temporali nel suo database [13].
È da puntualizzare che quest'ultimo non è persistente, Prometheus offre infatti un'interfaccia per copiare i suoi dati in un sistema di storage remoto [13].
I dati raccolti da Prometheus possono essere visualizzati con un tool di visualizzazione e dashboard come **Grafana**. Con Grafana è possibile creare dashboard interattive, visualizzare metriche di serie temporali, settare allarmi e notifiche e osservare dati in tempo reale.
Sia Prometheus che Grafana possono essere messi in esecuzione in un cluster come Pod, in un namespace dedicato.

5.9 Sicurezza

Istio fornisce diverse funzionalità per garantire sicurezza nelle comunicazioni tra microservizi definendo identità, policy di sicurezza, cifratura con mTLS, autenticazione, autorizzazione e audit (report delle attività).

Istio assegna un'**identità** univoca di tipo SPIFFE ID ad ogni workload, con certificati X.509. Di default viene usato uno strumento integrato chiamato Citadel come Certification Authority, ma si può usare anche Spire [12].

Istio propone due tipi di **autenticazione**: l'autenticazione service-to-service si ottiene con una connessione mTLS, mentre l'autenticazione di richieste degli utenti viene fatta tramite un JSON Web Token. Quest'ultimo viene fornito all'utente da un identity provider (es. Google Identity) [12].

Istio propone delle funzionalità di **autorizzazione** per definire che cosa possono fare le risorse di ogni Mesh, namespace e i relativi workload. Attraverso delle policy di autorizzazione (file YAML) si può stabilire quali richieste ad un servizio sono concesse. Queste policy vengono messe in pratica dai proxy Envoy [12].

5.10 Health Check

Come descritto nel paragrafo 2.4, Kubernetes utilizza dei meccanismi di **Liveness Probe** e **Readiness Probe** per controllare lo stato di salute di un container e la sua disponibilità a ricevere traffico. Questi due meccanismi dicono al kubelet del nodo di interrogare i container tramite un comando (apertura di un file del container che contiene l'informazione), una richiesta HTTP o gRPC oppure una richiesta di apertura di una socket TCP.

Per quanto riguarda l'interrogazione con un comando, i proxy Istio non fanno da intermediari per questo tipo di comunicazione con i Pod o container, perciò può essere utilizzato senza problemi.
Per il caso di HTTP e gRPC i proxy Istio intercettano il traffico e lo mettono in sicurezza con mTLS. Istio non rilascia un certificato ai kubelet, e quindi questi non possono interagire con i Pod o container tramite i protocolli e framework citati precedentemente. Perciò le richieste di Health Check non potrebbero funzionare. Per risolvere questo problema Istio ridefinisce la Liveness Probe prima di applicare il file di configurazione, cambiando il percorso a cui il kubelet dovrà interrogare il container. Il nuovo percorso a cui il kubelet farà la richiesta punta al proxy Istio, che lo tradurrà in quello giusto [12].
In seguito verrà mostrato un esempio di come viene cambiata la Liveness Probe del file di configurazione che viene applicato.

```
1 livenessProbe:
2   httpGet:
3     path: /foo
4     port: 8001
5   initialDelaySeconds: 5
6   periodSeconds: 5
```

Figura 5.7: Esempio di Liveness Probe che viene applicata

In Figura 5.7 si può vedere che la Liveness Probe indica al kubelet di interrogare il container con una richiesta HTTP ad un percorso interno allo stesso.

```
livenessProbe:
  httpGet:
    "path": "/app-health/liveness-http/livez",
    "port": 15020,
    "scheme": "HTTP"
  initialDelaySeconds: 5
  periodSeconds: 5
```

Figura 5.8: Liveness Probe di Figura 5.7 modificata da Istio

In Figura 5.8 si può vedere che la Liveness Probe applicata è stata cambiata, modificando il percorso con uno interno al proxy Istio. L'output mostrato in Figura 5.8 è stato ottenuto con il comando "kubectl get pod".

Per il caso di richieste di apertura di una socket TCP verso il container, il kubelet manda una richiesta ad una porta e se la trova aperta il risultato dello Health Check è positivo. Il problema è che il proxy Istio intercetta tutto il traffico TCP e quindi è in ascolto ad ogni porta, perciò il kubelet si connetterebbe sempre con successo e lo Health Check avrebbe sempre esito positivo.
In modo simile al caso di richieste HTTP e gRPC, Istio ridefinisce la Liveness Probe prima di applicare il file di configurazione, cambiando la porta che il kubelet interrogherà. Quando il proxy Istio riceverà richieste di connessione a questa nuova porta, farà lui stesso una richiesta di connessione alla corretta porta del container da monitorare, inoltrando il risultato al kubelet. In questo modo il kubelet può fare una richiesta di connessione al container in modo indiretto [12].

5.11 Istio per sistemi legacy

Istio è stato pensato per funzionare con servizi Kubernetes. Spesso convertire un servizio a Kubernetes può risultare complesso o poco conveniente, perciò Istio propone una soluzione per consentire l'integrazione di un sistema legacy con una Mesh Istio. I servizi upstream forniti da sistemi legacy possono essere raggiunti attraverso un Egress Gateway [12]. In Figura 5.9 viene mostrato un esempio di implementazione di questa soluzione, che verrà meglio illustrata in seguito.

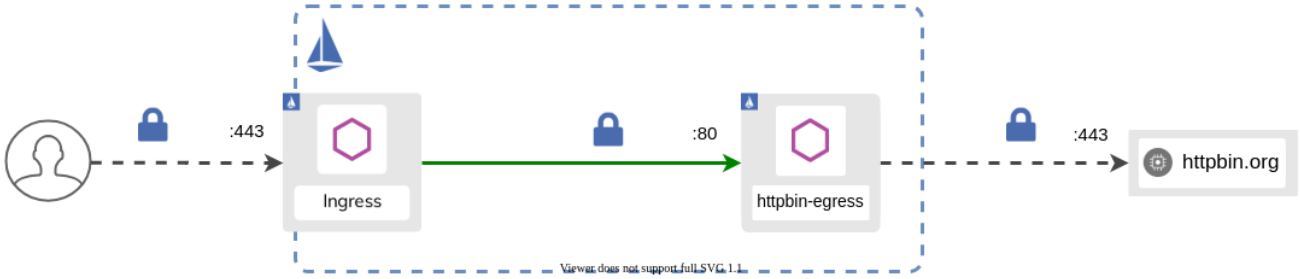

Figura 5.9: Esempio di integrazione di un sistema legagy con Istio [https://istio.io/latest/blog/2020/proxying-legacy-services-using-egress-gateways/proxying-legacy-services-using-egress-gateways-communication.svg]

Per rendere accessibile il servizio upstream a https://httpbin.org/ serve:

- Un **Egress Gateway** dedicato alle comunicazioni uscenti dalla Mesh verso il servizio
- Un **certificato TLS** per creare una connessione sicura fuori dal cluster con il servizio esterno
- Un **Ingress Gateway** per definire protocolli e porte accessibili da fuori la Mesh
- Un **Virtual Service** che dirotta il traffico proveniente da dentro la Mesh oppure dall'Ingress Gateway verso l'Egress Gateway e poi verso httpbin.org tramite HTTPS
- Un **Service Entry** per autorizzare la comunicazione al servizio esterno, specificando protocollo HTTPS, porta 443 e protocollo TLS

Per richiedere l'upstream httpbin.org da dentro un Pod si può semplicemente usare il comando curl dandogli come paramentro l'URL, la richiesta sarà intercettata dal sidecar e inoltrata prima all'Egress Gateway e poi al servizio stesso.
Per poter fare la stessa richiesta ma da fuori dal cluster bisogna prima fare port-forward della porta dell'Ingress Gateway, ovvero esporre la porta dedicata al Gateway all'esterno del cluster. Poi si può procedere usando il comando curl, specificando di voler risolvere l'hostname httpbin.org tramite localhost alla porta esposta dal cluster, invece che usando un DNS [12].

Capitolo 6: Network Service Mesh

Nel seguente capitolo verrà illustrata la tecnologia NSM, il contesto in cui può essere utilizzato, la sua architettura e il suo metodo di lavoro. Verrà posta particolare attenzione verso le funzionalità che offre in modo simile a Istio, come osservabilità, sicurezza e controllo della salute del sistema. Infine, verrà fatto un confronto dettagliato tra NSM e Istio.

Nel contesto di Kubernetes, il termine **workload** indica qualsiasi unità di lavoro computazionale che gira nel cluster. È un'applicazione o servizio che viene eseguito e gestito da Kubernetes.
In un ambiente Cloud Native, un'applicazione non è un monolite, ma è composta da più workload distribuiti (es. microservizi) che possono girare in Pod diversi, su namespace diversi, su nodi diversi, su cluster diversi, in cloud diversi (es. AWS, Google Cloud Platform).

Un **Runtime Domain** è un sistema dove sono in esecuzione i workload (es. cluster Kubernetes, cluster di VM, datacenter). Quando si definisce un Runtime Domain (ambiente dove far girare i workload), automaticamente gli si associa un unico dominio di connettività, una sua rete dedicata. Ogni workload può connettersi solo a un unico dominio di connettività (quello associato al runtime domain in cui gira). Solo i workload dentro uno specifico runtime domain possono far parte del suo dominio di connettività. È quindi critico il fatto di non poter far connettere un workload a una rete di un altro runtime domain, perchè per esempio i workload che fanno parte di una stessa applicazione hanno bisogno di comunicare a prescindere dall'ambiente in cui sono in esecuzione [14].
Per fare un esempio di questo accoppiamento tra Pod in esecuzione in ambienti diversi si può pensare a un'applicazione le cui componenti sono distribuite in cluster diversi: il back-end risiede su AWS mentre il front-end risiede su Google Cloud Platform. I Pod in esecuzione sul cluster del front-end hanno bisogno di comunicare con quelli sul cluster del back-end, ma i cluster non sono naturalmente interconnessi.

La tecnologia ***Network Service Mesh*** (NSM) nasce per consentire a questi workload di comunicare tra loro astraendo la rete. Li aiuta a trovare il servizio giusto, creare la connessione e farla funzionare a prescindere da dove i workload siano fisicamente eseguiti. NSM lavora in modo indipendente dal Runtime Domain. È compatibile con Kubernetes e non interferisce con il funzionamento del CNI del cluster [14].

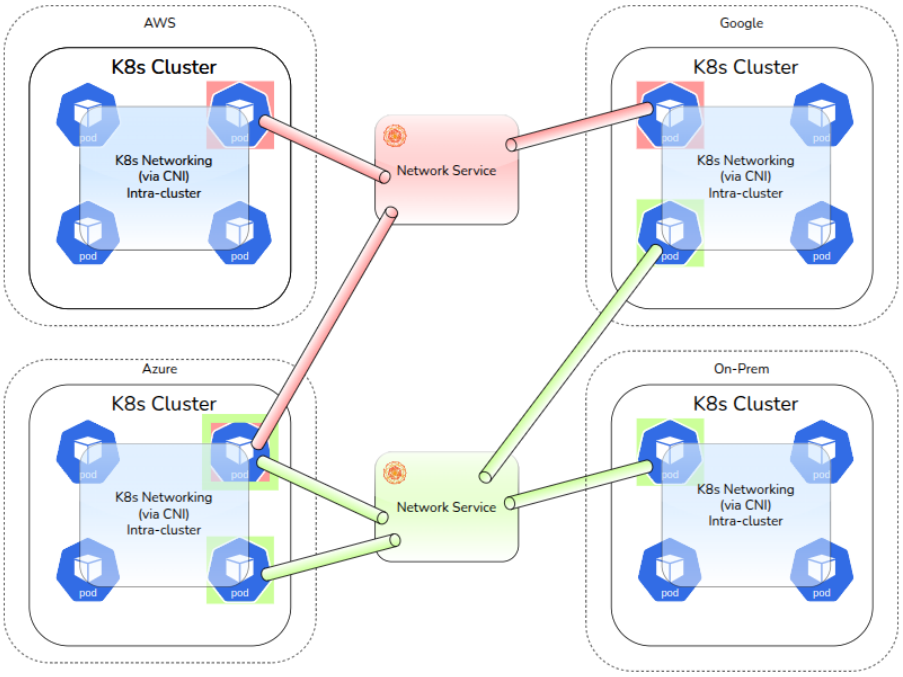

Figura 6.1: Esempio architettura di NSM [https://networkservicemesh.io/img/concepts/enterprise/nsm_solution.svg]

In Figura 6.1 viene mostrato uno schema di funzionamento di NSM, in cui Pod che risiedono in cluster diversi vengono connessi tra loro tramite un'infrastruttura NSM.

6.1 Architettura

Tutte le risorse utilizzate da NSM sono create mediante Custom Resources di Kubernetes.

Un **Network Service** può essere definito come un insieme di funzionalità di Connettività, Sicurezza e Osservabilità, applicate a livello L3 e superiori del modello OSI, alle quali i workload possono connettersi in modo individuale. Consente a un Pod all'interno di un cluster di connettersi ad una intranet in modo sicuro ed evitando complicazioni quali configurazione di interfacce, sottoreti e instradamento.
In NSM l'unità di connessione non è più solo una porta TCP, ma un canale di traffico incapsulato, che può attraversare catene di funzioni di rete (firewall, IDS, proxy, overlay VPN, ecc). Questo canale viene chiamato payload, perché descrive cosa viene trasportato, non su quale porta [14].

Un client in NSM, chiamato anche **Network Service Client** (NSC), è un workload che richiede di essere connesso a un Network Service tramite il suo nome. Viene autenticato tramite Spiffe ID (nel formato: spiffe://<trust-domain>/<path>) e deve essere autorizzato per poter essere collegato a un determinato Network Service. Un client può essere un Pod, una macchina virtuale

o un server fisico [14].

Un endpoint in NSM, a volte chiamato anche **Network Service Endpoint**, è l'entità che fornisce il Network Service al client. Può essere un Pod nello stesso cluster o in un cluster differente rispetto al client, una macchina virtuale o in generale un componente della rete fisica [14].

Un ***vWire*** (o Virtual Wire) agisce come cavo virtuale tra un client (Pod) e un endpoint (NSE). Ogni vWire ha le seguenti caratteristiche:

- I pacchetti che entrano nel vWire dal client usciranno all'endpoint, e viceversa
- Solo i pacchetti che sono entrati nella vWire dal lato del Client usciranno dal lato dell'Endpoint, e viceversa
- Un Endpoint può avere più vWire (connessioni virtuali in ingresso).
- Un Client può avere più vWire (connessioni virtuali in uscita).
- Ogni vWire trasporta traffico per un solo Network Service

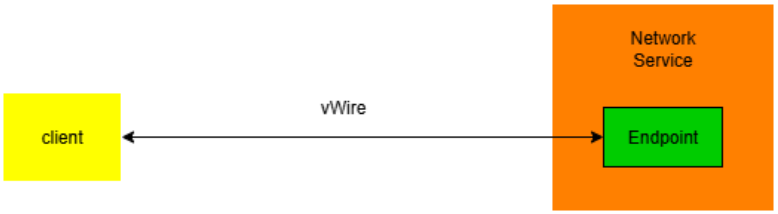

Figura 6.2: Architettura di un vWire di NSM

Quando un Pod richiede una connessione con un NSE, NSM crea una virtual network interface dentro il Pod. In questo modo il Pod può mandare e ricevere pacchetti dal link NSM.
Questa interfaccia appare solo quando il Pod richiede un servizio, solo il Pod stesso ha accesso a questo tunnel che viene creato in automatico da NSM, non c'è bisogno di configurazione manuale di una VPN all'interno del Pod.
Il VPN gateway del NSE conosce quali IP sono validi per il tunnel e fornisce al Pod un indirizzo IP (o MAC) e istruzioni per l'instradamento [14].

6.2 Network Service API

Il ***Google Remote Procedure Call*** (gRPC) è un framework open source sviluppato da Google per chiamate remote, utilizza Protocol Buffers (protobuf) per la serializzazione dei dati e HTTP/2 per trasmettere i dati.

Il client crea un vWire con il Network Service tramite una 'request' gRPC al NSE. Il vWire avrà un tempo di scadenza che può essere prolungato con nuove gRPC di tipo 'request' da parte del client. Un client può usare una chiamata gRPC di tipo "MonitorConnection" con streaming verso NSM per ricevere aggiornamenti

sullo stato di una vWire che ha con un Network Service [14].

Come avviene in qualsiasi altra Mesh, NSM dispone di **Registri** dei Servizi di Rete (NSR), nei quali vengono registrati i Network Service e i Network Service Endpoint. Un Network Service Endpoint fornisce uno o più Network Service, si registra nel Registry specificando l'elenco di Network Service che offre e le destination labels (etichette di destinazione) che sta pubblicizzando per ciascun servizio [14].

Ogni Network Service è identificato da un nome ed è registrato nel Network Service Registry. Ogni Network Service può specificare una lista di "match" (corrispondenze). Queste corrispondenze permettono di associare le "source labels" (etichette di origine) inviate da un Client nella sua richiesta, con le "destination labels" pubblicizzate da un Endpoint al momento della registrazione come fornitore del servizio [14].

```yaml
apiVersion: networkservicemesh.io/v1
kind: NetworkService
metadata:
  name: service-mesh@finance.example.com
spec:
  payload: IP
  matches:
    - source_selector:
        service: envoy-proxy
      routes:
        - destination_selector:
            service: vl3
    - source_selector:
      routes:
      - destination_selector:
          service: envoy-proxy
```

Figura 6.3: Esempio di file di configurazione di un NetworkService

Il file YAML in Figura 6.3 definisce un Network Service che sarà raggiungibile con una connessione L3, utilizzando pacchetti IP. Se un workload richiede il Network Service e fornisce nella sua richiesta una label uguale a "envoy-proxy", allora NSM deve reindirizzarlo ad un workload che espone un servizio nominato "service-mesh" e ha label service uguale a "vl3".
Gli altri workload saranno reindirizzati verso un workload che espone un servizio chiamato "service-mesh" e ha label service uguale a "envoy-proxy".

NSM consente l'esistenza di molteplici domini di registro indipendenti, che non hanno conoscenza reciproca. Il dominio di registro di un Network Service viene indicato aggiungendo un suffisso '@dominio' al nome del servizio. Quindi, per esempio, un Network Service chiamato "service-mesh" che si trova nel dominio "finance.example.com" sarà identificato come "service-mesh@finance.example.com" [14].

Il Registry Server viene rintracciato richiedendo al DNS (di solito quello di Kubernetes) una entry SRV, quindi il dominio e la porta del servizio.

A volte un Network Service è fornito da un grafo di Endpoint composti insieme per servire un certo workload. Ad esempio, quando si vuole offrire un Traditional Service Mesh come Network Service, può risultare più semplice comporre un proxy Envoy (gestito da un Control Plane come Istio) con un vl3, che fornisce un dominio virtuale di livello 3 (L3) [14].

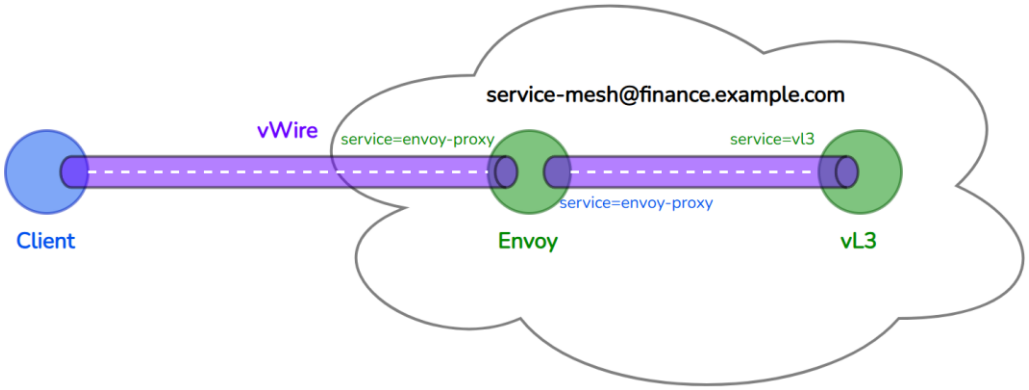

Figura 6.4: Esempio di utilizzo di un dominio di registro di un Network Service [https://networkservicemesh.io/img/concepts/architecture/composition.svg]

In Figura 6.4 mostra un client che si connette ad un Network Service chiamato "service-mesh", nel dominio "finance.example.com". La richiesta è reindirizzata ad un service "vl3"

6.3 Componenti

NSM utilizza un componente chiamato **Network Service Manager** (NSMgr), che è responsabile di fare discovery (scansionare la rete) dei Network Services e Network Service Endpoints e dell'elaborazione delle richieste dei client. Deve essere situato sulla stessa macchina del client NSM per fornire una connessione alla rete NSM, perciò è una agent in esecuzione in ogni nodo (DaemonSet). Può anche fungere da registry se non esiste un registry vero e proprio [14].
Quando un client richiede l'utilizzo di un Network Service da parte dei Pod i NSMgr trova e instaura una connessione con il NSE e crea/gestisce il tunnel virtuale con lo stesso. Comunica con i Pod tramite gRPC e inietta una connessione con il NSE richiesto dentro al Pod.

Un **datapath provider** è un'entità responsabile di gestire il traffico di rete tra le applicazioni e i servizi, garantendo che i dati vengano recapitati correttamente. Inoltre può fornire funzionalità di rete avanzate, come ad esempio la gestione delle politiche di sicurezza, la qualità del servizio (QoS) e la visibilità del traffico. Uno dei datapath provider di NSM si chiama **Forwarder**, si occupa di

configurare le interfacce di rete richieste dai client NSM. È usato da NSMgr per creare l'interfaccia di connessione sia sul client che sull'endpoint [14].

NSM utilizza un **Registry** per registrare i Network Service e i NSE [14].

6.4 Esempio di caso d'uso di NSM

Si vuole sviluppare un applicazione per Kubernetes da distribuire in cloud. Uno dei Pod dell'applicazione ha bisogno di accedere ad una intranet aziendale (sistema legacy), sorgono quindi delle criticità nel configurare VPN gateways, routing tables, NAT, firewall rules.
Per esempio:

- L'IP del Pod è dinamico, quindi non si possono applicare regole di routing o di firewall statiche
- Bisogna configurare instradamento per ogni IP della intranet, ma se questi cambiano si è costretti a riconfigurarli da capo
- Se nella intranet viene aggiunto un nuovo CIDR bisogna configurare le regole di instradamento

Una semplice soluzione a questo problema, usando un Network Service di NSM, può essere:

```yaml
kind: NetworkService
apiVersion: V1
metadata:
  name: secure-intranet-connectivity
spec:
  selector:
    app: secure-intranet-gateway
    payload: IP
```

Figura 6.5: NetworkService

Questo file di configurazione .yaml definisce un tipo di connessione che un Pod può richiedere. "secure-intranet-connectivity" è il nome del Network Service, in spec.selector si specifica quale implementazione del NSE dovrà essere chiamato da NSM, ovvero chi fornisce il servizio. In questo caso il payload utilizzato è IP (L3), quindi il network service farà tunnelling di pacchetti IP (invece che frames Ethernet di L2). "secure-intranet-gateway" è la label dell'applicazione usata per trovare i Pod che fanno da gateway verso la rete intranet [14].

6.5 Instaurazione di una connessione tra Pod

Di seguito vengono elencati i principali passaggi messi in pratica in un'infrastruttura NSM al fine di instaurare una connessione tra due Pod [14]:

- Un Pod vuole esporre un Network Service, quindi avvia un NSE che informa il NSMgr del nodo
- Il NSMgr richiede all'API Registry (il Control Plane di NSM) di registrare

un NSE
- Quando un altro Pod vuole richiedere una connessione con quel Network Service chiede al NSMgr locale di interrogare l'API Registry per trovare il relativo NSE
- A questo punto il NSMgr del Pod client instaura una connessione con il NSMgr del Pod server
- I Forwarder su entrambi i nodi creano una connessione virtuale (es. VXLAN, kernel veth, memif, ecc.)
- Questa connessione viene poi iniettata prima nel Pod server e poi nel Pod client

6.6 Tipi di configurazioni della connessione tra Pod

NSM propone diversi tipi di configurazioni per l'instaurazione di una connessione tra Pod. Ogni configurazione utilizza tecnologie diverse per le interfacce di rete, la virtualizzazione, il payload.
Per la parte sperimentale di questa tesi verranno usate configurazioni "vl3" e "kernel to kernel", che verranno introdotte in seguito.

Una configurazione **kernel to kernel** di NSM usufruisce di una tecnologia chiamata VXLAN.

Virtual eXtensible LAN (VXLAN) è un metodo di virtualizzazione della rete che consiste nell'incapsulare frame Ethernet (L2) in datagrammi UDP (L4). In questo modo consente di collegare alla stessa VLAN sistemi remoti [15].

In una configurazione **kernel to kernel** di NSM client ed endpoint hanno un'interfaccia virtual ethernet. Questa viene creata dal NSMgr del nodo attraverso comandi Linux. Client ed endpoint comunicano attraverso un tunnel VXLAN creato e gestito da un Forwarder, in cui i frame vengono incapsulati e instradati nella rete Kubernetes.

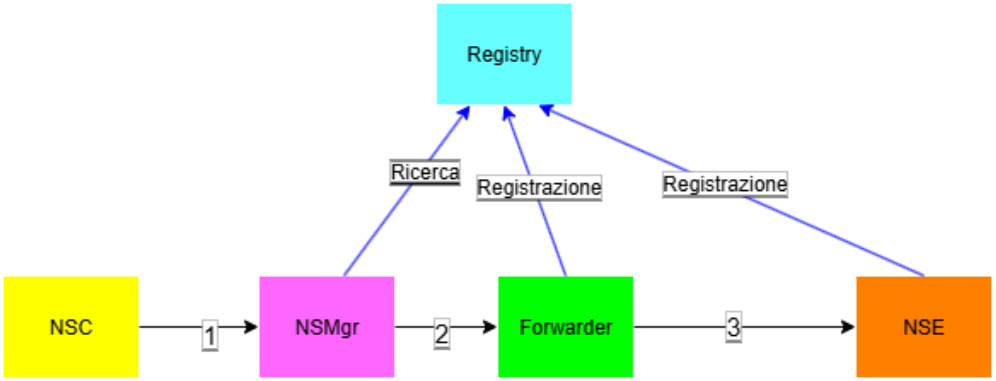

Figura 6.6: Meccanismo con cui il NSC trova il NSE, entrambe in uno stesso cluster, configurazione k2k

In Figura 6.6 è mostrato come un Network Service Client riesce a ritracciare un Network Service Endpoint. Il NSC fa una richiesta al NSMgr del nodo, questo poi interroga il Registry che gli dice dove si trova il NSE. In questo caso il NSE è nello stesso nodo, perciò il NSMgr non deve interrogare altri NSMgr ma non gli rimane che contattare il Forwarder del nodo e chiedergli di instaurare una connessione tra NSC e NSE.
In Figura 6.6 viene anche mostrato che sia il Forwarder che il NSE devono registrarsi nel Registry, così da potere essere rintracciati.

Una configurazione **vl3** di NSM usufruisce di tecnologie quali VPP, interfacce memif, IPAM.

Ogni dispositivo della rete riceve, ispeziona e inoltra pacchetti. L'elaborazione dei pacchetti a livello di kernel segue un approccio sequenziale: un singolo pacchetto viene ricevuto, quindi il processore preleva una per una le istruzioni dalla cache delle istruzioni e le esegue sul pacchetto. Al termine, il pacchetto prosegue il proprio percorso nella rete (forwarding), e solo allora il pacchetto successivo entra nel medesimo ciclo di elaborazione.
Il ***Vector Packet Processing*** (VPP) è un diverso approccio per l'elaborazione dei pacchetti: questa viene fatta non nello spazio del kernel ma nello spazio dell'utente, dove le risorse sono meno limitate. VPP processa gruppi di pacchetti insieme, offre migliore throughput e minore latenza [16].

VPP utilizza interfacce ***memif*** per connettere due processi. memif è un interfaccia virtuale a memoria condivisa. Invece che inviare pacchetti attraverso socket o classiche interfacce, memif crea una regione di memoria in cui entrambi gli endpoint possono accedere e inoltrare pacchetti [15].

IP Address Managment (IPAM) è un software per gestire, e monitorare lo spazio degli indirizzi IP in una rete. Assegna e reclama indirizzi IP, gestisce sottoreti e VLAN, integra servizi di DHCP e DNS [15].

In una configurazione **vl3** di NSM un NSE crea una rete IP virtuale e gestisce gli indirizzi IP tramite IPAM. Quando un client richiede un servizio, il NSE gli assegna un IP della sottorete. VPP è il Data Plane di questa configurazione e si occupa di inoltrare i pacchetti tra client ed endpoint. VPP gestisce interfacce memif, che connettono i client con i Forwarder basati su VPP.
Due Pod con un'interfaccia memif sono connessi con una socket unix tramite un volume condiviso da Kubernetes.

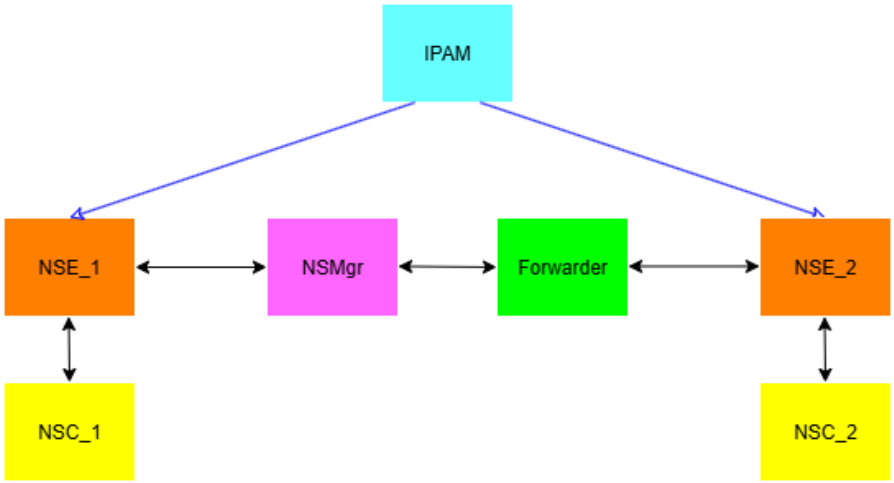

Figura 6.7: Connessione tra due NSC, in uno stesso cluster, configurazione vl3

In Figura 6.7 viene mostrato come due NSC comunicano in una configurazione vl3 di NSM. Nell'esempio i due NSE si trovano nello stesso nodo, perciò quando NSC_1 richiede una connessione con NSE_2, il NSMgr del nodo contatta il Forwarder per chiedergli di instaurare una connessione.

NSM propone anche altri tipi di configurazioni, che però non verranno introdotte in questa tesi in quanto esulano dagli obiettivi di quest'ultima. Queste configurazioni utilizzano altre tecnologie di virtualizzazione, quali VFIO e SR-IOV.

6.7 Osservabilità

NSM offre funzionalità di osservabilità servendosi della tecnologia ***OpenTelemetry***.

OpenTelemetry è un framework gestito dalla Cloud Native Computing Foundation (CNCF) che serve a generare, processare e trasmettere tipi differenti di dati telemetrici e segnali. Il sistema da osservare deve generare segnali in modo automatico oppure attraverso il SDK di OpenTelemetry. I dati generati dal sistema devono essere conformi allo Open Telemetry Protocol, questo descrive proprietà del sistema che vengono osservate. I dati possono essere tracce, metrice

oppure log [17].

Le **tracce** rappresentano il percorso di una richiesta all'interno del sistema, ovvero quali componenti la gestiscono e in che ordine. Sono una collezione di span, che sono operazioni come chiamate ad un database, richieste http o chiamate a funzione ed hanno diversi attributi quali status e timestamp [17].

Le **metriche** sono dati numerici legati ad un timestamp e rigurdano il funzionamento di un servizio oppure l'utilizzo delle risorse (es. CPU e memoria) [17].

I **log** sono record di testo legati ad un timestamp che provengono da un componente dei sistema [17].

Questi dati vengono mandati allo ***Open Telemetry Collector***, che si occupa di filtrare queste informazioni o comporle insieme (unire informazioni provenienti da fonti diverse). Questi dati modificati vengono poi inoltrati ad altri collector o a tool di visualizzazione come Prometheus, Jaeger, Grafana [17].

I componenti di NSM, quali NSMgr, Registry, Forwarder ecc. utilizzano il SDK di OpenTelemetry per generare segnali. Questi possono essere collezionati e visualizzati con Prometheus e Jaeger, oppure con la Dashboard di NSM.

6.8 Sicurezza

NSM adotta l'approccio ***Zero Trust*** come strategia di sicurezza e utilizza ***SPIFFE ID*** per l'autenticazione dei workload.

Per implementare SPIFFE serve uno **SPIRE Server** (SPIFFE Runtime Environment).
NSM usa SPIFFE per autenticare i workload a livello di rete (L3) e applicazioni che usano API o richieste HTTP (L7).

Un deployment SPIRE è costituito da un SPIRE Server e da uno o più SPIRE Agent. Lo SPIRE Server funge da autorità di firma per le identità rilasciate ai workload, mediante gli agent distribuiti. Inoltre, mantiene un registro delle identità dei workload e delle condizioni che devono essere verificate affinché tali identità possano essere emesse [6].

Gli SPIRE Agent, invece, espongono localmente l'API SPIFFE Workload ai workload presenti sul nodo in cui sono installati. È necessario installare un agente su ciascun nodo in cui venga eseguito un workload che richieda un'identità

SPIFFE [6].

6.9 Healing e resilienza

Le proprietà di Healing servono a garantire disponibilità, affidabilità e resilienza al lavoro dei workload che utilizzano NSM. I Network Service Clients usano delle API per monitorare il NSMgr del nodo e la connessione con il Network Service. Se avviene un cambiamento di stato della connessione (es. non è più Ready), oppure il NSMgr elimina il link al servizio o chiude il canale di comunicazione, allora il client richiede di nuovo il Network Service per instaurare una nuova connessione. Quindi, attraverso le API di monitoraggio, il client si può accorgere del fallimento del Control Plane (es. il Pod NSMgr smette di funzionare) e del fallimento del Data Plane (es. VPP smette di funzionare) [14].

6.10 Cooperazione tra Service Mesh e NSM

Service Mesh classici e NSM operano a diversi livelli dello stack di rete, ma possono cooperare in architetture complesse. Un Service Mesh tradizionale (come Istio) può essere modellato come un servizio di rete (Network Service) e quindi orchestrato usando NSM. NSM quindi tratta ogni Service Mesh come un endpoint di rete. Se per esempio un'applicazione su Cluster B vuole usare funzionalità di osservabilità fornite da Istio su Cluster A con NSM si può creare una connessione IP dinamica dal Pod su B al Service Mesh su A, senza dover installare Istio su B [14].

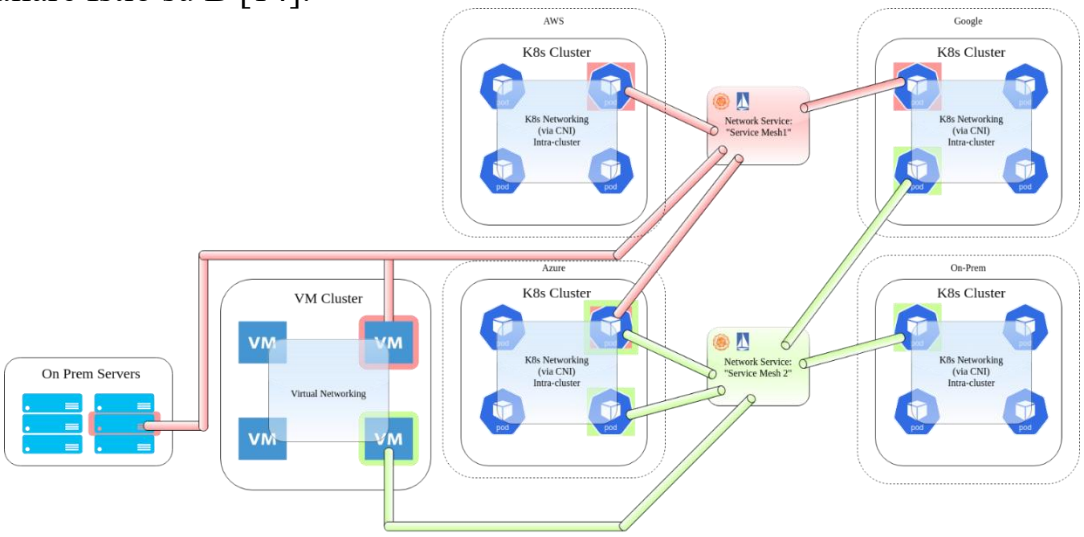

Figura 6.8: Esempio cooperazione tra Istio e NSM
[https://networkservicemesh.io/img/concepts/enterprise/traditional_service_mesh.svg]

In Figura 6.8 viene mostrato un esempio di cooperazione tra Service Mesh tradizionali e NSM. Ogni Pod viene trattato come un endpoint di rete e inserito nell'infrastruttura NSM, così che potrà contattare altri Pod in cluster diversi.

6.11 Confronto tra Istio e NSM

In questa sezione verrà fatto un confronto dettagliato tra Istio e NSM con l'obiettivo di sottolineare quali funzionalità hanno in comune e quali no, e come le implementano.

	Istio	Network Service Mesh
Aspetti legali		
Fondatori	Google, IBM, Lyft	CNCF (progetto sandbox)
Open source	Si	Si
Licenza	Apache License 2.0	Apache License 2.0
Tipo di Mesh	Service Mesh	Network Mesh
Architettura		
Livello OSI	Livello 4, Livello 7	Livelli 2 e 3
Tecnologie usate	Envoy Proxy, gRPC, mTLS, Iptables	VXLAN, VPP, VFIO, SR-IOV, Iptables, Linux Kernel Networking, gRPC
Protocolli principali	HTTP, HTTPS, TCP	TCP
Tipo di comunicazione	Comunicazione tra microservizi (applicazioni)	Connessione di rete tra workload, Pod e sistemi esterni al cluster
Funzioni principali	Routing L7, bilanciamento carico, sicurezza, retry, circuit breaking, telemetria	Provisioning dinamico di servizi di rete, segmentazione L2/L3, networking multicloud e legacy, sicurezza, telemetria
Gestione del traffico	Retry, timeouts, gestione del carico, circuit breaking	Solo a Livello 2/3: no retry a livello HTTP e TCP, no gestione del carico
Architettura	sidecar proxy Envoy (Data Plane), Control Plane	Modello client-endpoint, Control plane, Data plane
Integrazione con Kubernetes	Alta: CRD native, compatibilità con CNI e Ingress Controller, osservabilità sul cluster	Alta: CRD native, non interferisce con il CNI
Sicurezza	mTLS tra servizi, identità SPIFFE per i workload	Identità SPIFFE per i workload
Certification Authority	Citadel. Compatibile con Spire o altre.	Spire
Health Check	Liveness Probe, Readiness Probe	NSC monitora la connessione
Complessità di implementazione	Moderata	Alta, soprattutto in ambienti complessi o con reti personalizzate
Chi espone le metriche	Control Plane, Proxy Envoy, Gateways	Control Plane, Forwarder, NSMgr, Registry, NSE, NSC
Osservabilità	Prometheus, Grafana, Kiali, Jaeger	Prometheus, Jaeger, Dashboard di NSM
Caso d'uso tipico	Architetture a microservizi, gestione traffico applicativo	Integrazione di sistemi legacy, ambienti ibridi/Cloud Native

Tabella 6.1: Confronto tra Istio e NSM

Istio lavora con messaggi applicativi di L7, che sono i suoi payload. Gestisce quindi richieste e risposte **HTTPS** e fa routing basato per esempio su URI o autenticazione TLS. I protocolli sottostanti (Ethernet, IP, TCP) possono essere modificati o completamente rimpiazzati.
NSM offre funzionalità simili ma a L3: il payload gestito è l'intero **pacchetto IP** o il **frame Ethernet**, client ed endpoint comunicano all'interno di una stessa sottorete.

Istio adotta il modello **sidecar proxy**, mettendo in esecuzione in ogni Pod un istanza del **proxy Envoy**. I suoi microservizi comunicano attraverso richieste HTTP, gRPC e connessioni TCP. Viene usato mTLS per garantire sicurezza tra le comunicazioni.
A seconda del caso applicativo, NSM può usare diverse tecnologie per instaurare una connessione tra client ed endpoint: **VXLAN**, **VFIO**, **SR-IOV**. Vengono poi utilizzate le funzionalità di Linux Kernel Networking per creare delle interfacce nei Pod. Possono essere usate interfacce memif e sistemi VPP.

Istio viene più comunemente usato per connessioni tra microservizi: il funzionamento delle applicazioni dipende dal buon funzionamento dell'interazione tra microservizi. Per fare ciò implementa meccanismi di gestione del carico, retry, circuit breaking, timeout.
NSM si occupa di **connessioni tra client ed endpoint a Livello 2 e 3**. Gli endpoint possono essere sistemi legacy: NSM risolve il problema di come integrarli in un cluster. NSM non si occupa di gestione del traffico: si occupa solo della comunicazione a Livello 2 e 3. Non ci sono meccanismi di gestione del carico, non fa ritrasmissioni.

Envoy è il Data Plane di Istio: intercetta e gestisce il traffico tra Pod applicando le regole definite dall'amministratore di rete. Il Control Plane di Istio coordina il funzionamento di ogni proxy Envoy e si occupa di Service Discovery, sicurezza, e raccoglie dati telemetrici da fornire a Prometheus.
In NSM il Data Plane è il **Forwarder** del nodo, che fa da intermediario tra i due capi di ogni connessione e utilizza la tecnologia VPP. Il Control Plane di NSM comprende il NSMgr del nodo e il Registry, si occupa di ricevere le richieste di connessione e discovery da parte dei client. Deve quindi interfacciarsi con altri NSMgr del cluster e coordinarsi con loro per l'instaurazione della connessione.

Sia Istio che NSM estendono le API di Kubernetes utilizzando delle **Custom Resource Definition** (CRD). Istio è compatibile con **CNI** e si serve degli indirizzi IP che quest'ultimo assegna ai Pod. NSM funziona a prescindere dal **CNI**, che non è rilevante per il suo funzionamento. Infatti, NSM si occupa di instaurare una connessione tra Pod, è lui a creare connettività e fornire gli indirizzi IP.

Sia Istio che NSM assegnano ad ogni workload un'identità **SPIFFE**. Istio mette in sicurezza le comunicazioni tra Pod tramite **mTLS**.
La Certification Authority usata da Istio è **Citadel**, un componente creato apposta per il progetto che fa parte del Control Plane istiod. È possibile usare **Spire** o altre, che sono da specificare durante l'installazione di Istio.
NSM invece utilizza **Spire**. Per fini di test è anche possibile non usare nessuna Certification Authority. È assolutamente sconsigliato non farne uso in ambienti di produzione.

Il Registry di Istio è composto da **tutti i servizi della piattaforma** in cui opera (es. Kubernetes) e i servizi dichiarati in ogni risorsa **Service Entry**. Oltre agli indirizzi IP dei servizi e degli endpoint contiene altre informazioni come le identità SPIFFE e i dati delle risorse di Istio (es. Virtual Service, Destination Rule ecc.).
In NSM invece, il Registry contiene solo l'elenco dei **Network Servic**e disponibili e i relativi NSE. I servizi vengono divisi in domini.

Il kubelet di Kubernetes ha il compito di interrogare i container per verificare il loro stato di salute e se sono pronti a ricevere traffico. Le modalità di questa verifica sono specificate in **Liveness Probe** e **Readiness Probe**. Quando un Pod contiene un sidecar proxy Envoy, questo intercetta tutto il traffico, perciò il kubelet non può interrogare direttamente i container. Quindi Istio **ridefinisce** Liveness Probe e Readiness Probe quando viene applicato un file di configurazione, così da permettere al kubelet di interrogare indirettamente i container, facendo una richiesta al proxy Envoy.
Per quanto riguarda NSM, i meccanismi di Health Check che introduce vengono messi in pratica dai client: questi **monitorano la connessione** e ne richiedono una nuova in caso di problemi.

La complessità di installazione di Istio non è eccessiva: si tratta di installare il Pod del suo Control Plane nel cluster e abilitare la **istio-injection** in ogni namespace in cui lo si vuole utilizzare. A questo punto ogni Pod del namespace conterrà un container con un'istanza del proxy Envoy. Tutti questi proxy saranno quindi già pronti per intercettare il traffico e comunicare tra loro. Si può quindi procedere con configurare VirtualService, DestinationRules e Gateways per gestire il traffico di più Pod/applicazioni.
La complessità di installazione di NSM è più elevata: dopo aver installato il Control Plane e Spire bisogna decidere che tipo di interfacce e metodo di virtualizzazione usare per connettere i Pod. NSM propone diverse configurazioni pronte all'uso, come **vl3** e **kernel to kernel**, però si può anche usare un template per creare un modello di comunicazione personalizzato. Il template si può trovare sulla pagina di GitHub del progetto NSM, è completo solo di Dockerfile e qualche file di configurazione YAML, il codice main.go è da scrivere interamente.

Con Istio i dati e le metriche vengono esposte dal Control Plane, da tutti i proxy Envoy e dai Gateways in un formato compatibile con **Prometheus**. Quest'ultimo li raccoglie e li può fornire a tool di visualizzazione e dashboard quali Grafana e Jaeger.
In NSM ad esporre le metriche sono il Control Plane, i Forwarder, i NSMgr, i Registry, i NSE e i NSC. Questi componenti utilizzano il SDK di **OpenTelemetry** per emettere metriche compatibili con il suo protocollo. Nel repository di GitHub di NSM è presente una configurazione pronta all'uso di Prometheus e Jaeger per la visualizzazione dei dati. Esiste anche una **dashboard** nativa di NSM.

Il caso d'uso tipico di Istio è la gestione delle comunicazioni e del traffico tra microservizi, all'interno di uno o più cluster. NSM invece è stato pensato per risolvere il problema dell'integrazione di sistemi legacy in un cluster. Può però essere utilizzato come semplice Service Mesh, gestendo quindi la comunicazione tra Pod e microservizi.

I **sistemi legacy** (es. Oracle DB) usano protocolli di comunicazione personalizzati, spesso ottimizzati per velocità o affidabilità, e si basano sulle caratteristiche del protocollo IP (es. indirizzi, routing, multicast). Per questi sistemi un classico Service Mesh L7 non sarebbe adatto perchè non è capace di utilizzare protocolli L3.
Istio può essere usato per connettere ad un cluster un sistema legacy solo se supporta HTTP e TCP.
Le differenze tra utilizzo di Istio e NSM per interagire con sistemi legacy vengono riassunte nella seguente tabella:

Caratteristica	Istio	Network Service Mesh
Livello OSI	Livello 7	Livello 2/3
Tipo di connessione	HTTP, gRPC, TCP mediante proxy	IP, Virtual Ethernet
Caso d'uso	Applicazioni legacy che supportano HTTP e TCP e possono usare un proxy	Sistemi legacy che hanno bisogno di creare una connessione (es. IP, VLAN, bridge L2)
Sicurezza	mTLS e SPIFFE ID a L7	SPIFFE ID a L3
Come si instaura la connessione	sidecar proxy (Envoy) in Kubernetes	Interfacce virtuali e tunnel tra workload e/o sistemi esterni
Protocolli supportati	HTTP, HTTP/2, gRPC, TCP	IP, Ethernet (e i protocolli che si basano su quest'ultimi)
Osservabilità	Metriche avanzate, tracciamento, telemetria	Semplice monitoraggio della connessione

Tabella 6.2: Confronto tra Istio e NSM utilizzati per sistemi legacy

Istio può integrare in un cluster solo sistemi legacy che supportano i protocolli **HTTP** o **TCP**, usando un proxy Envoy per inoltrare loro le richieste. NSM invece

si occupa anche di **instaurare una nuova connessione** a Livello 2 o 3, usando tecnologie di virtualizzazione della rete.
I proxy Envoy intercettano tutto il traffico tra cluster e sistema legacy e possono quindi fornire metriche avanzate. Invece NSM offre un semplice monitoraggio della connessione con il sistema legacy, in quanto quest'ultimo non espone metriche.

In questo capitolo è stata illustrata la tecnologia NSM e infine è stata fatta un'analisi comparativa delle tecnologie Istio e NSM. Nel capitolo successivo verrà spiegato quali test sono stati svolti per confrontare le due tecnologie in ambiti di utilizzo comuni; verranno poi esposti i risultati sperimentali raggiunti.

Capitolo 7: Test di confronto tra K3s semplice, Istio e NSM

L'obiettivo della parte sperimentale di questa tesi è stato quello di confrontare il funzionamento di Istio e NSM come Service Mesh, per vedere come si comportano in utilizzi diversi. In particolare, si è valutato il comportamento delle due tecnologie quando sottoposte a diverse richieste HTTP, TCP e gRPC. Successivamente sono stati fatti test sottoponendo le due tecnologie ad un carico variabile e infine al fallimento del Data Plane. È anche stato misurato l'utilizzo di risorse dei Control Plane di Istio e NSM e il tempo di avvio dei Pod delle due tecnologie.

TCP è il protocollo più utilizzato in Kubernetes, ogni Pod ha un indirizzo IP mediante il quale comunica con altri processi Linux. I proxy di Istio intercettano e gestiscono il traffico TCP di ogni Pod, mentre le comunicazioni tra Pod in NSM si basano su TCP. È perciò utile verificare come Istio e NSM performano con questo protocollo, in comunicazioni tra Pod.

I Pod in Kubernetes usufruiscono di TCP per comunicazioni con **HTTP** e **gRPC**. I Client raggiungono i servizi, per esempio i web server, con richieste HTTP o gRPC. Inoltre, gRPC è il protocollo usato da NSM per le comunicazioni tra Pod che fanno da NSMgr, Forwarder e NSE/NSC, al fine di gestire le connessioni virtuali. È quindi utile verificare le prestazioni di Istio e NSM quando due Pod richiedono o offrono servizi tramite HTTP e gRPC.

Un servizio in Kubernetes può essere sottoposto a diversi tipi di traffico e le sue prestazioni potrebbero variare. È stato quindi valutato il comportamento di Istio e NSM quando sottoposti a **carichi di tipo diverso**: carico esponenziale, lineare e Gaussiano.
Il carico esponenziale rappresenta il caso in cui avviene un improvviso e veloce aumento delle richieste ad un servizio. Un carico lineare avviene quando il traffico aumenta in modo costante. Infine, il carico Gaussiano può rappresentare, per esempio, la differenza di numero di richieste che un servizio riceve dai clienti tra il giorno e la notte.

Il **fallimento del Data Plane** è tra i peggiori imprevisti che possono capitare in una comunicazione tra Pod, in quanto si occupa della connettività tra gli stessi. È quindi utile verificare cosa succede se il servizio del Data Plane viene interrotto durante la comunicazione tra due Pod.

L'ambiente di lavoro e test è stato un cluster K3s formato da due macchine

virtuali Ubuntu Server che hanno fatto da nodi, uno Master e uno Worker. È stato scelto K3s come distribuzione di Kubernetes in quanto offre tutte le funzionalità necessarie per i test senza richiedere un elevato consumo di risorse. Ciò è stato ideale visto che i test sono stati svolti in un computer con potenza non elevata: processore AMD Ryzen 5 3500U (4 Core, 8 Thread) e 20GB di Ram.
Nel cluster sono stati installati Istio e NSM nelle loro configurazioni standard.
Per fare i test si è deciso di predisporre e confrontare quattro configurazioni del cluster.
In particolare, la prima configurazione è stata il cluster semplice K3s.

Nella seconda configurazione è stata abilitata la **istio-injection** nel namespace in cui risiedono i Pod del test, quindi per ognuno di questi Istio ha creato un side-car proxy che ha fatto da intermediario per le comunicazioni.

Nella terza configurazione è stata usata una connessione **kernel to kernel** (k2k) di NSM. È stata iniettata nel Pod una interfaccia kernel nel suo spazio dei nomi di rete, collegata ad un servizio di rete chiamato "kernel2kernel".
Vengono di seguito elencati i passaggi per l'instaurazione della connessione:

- NSM intercetta la richiesta di connessione del NSC al kernel-service
- Il NSMgr del nodo trova un NSE che fornisce il servizio
- NSM inietta dinamicamente un'interfaccia kernel in entrambi i Pod (NSC e NSE) e configura una connessione L2 punto-punto tra i due (cavo Ethernet virtuale)
- Vengono configurati indirizzi IP e regole di routing

Nella quarta configurazione è stata usata una connessione **vl3** (a livello 3), l'instradamento è stato fatto tramite indirizzo IP, quindi i client non hanno avuto bisogno di essere nella stessa sottorete o rete fisica. Un NSE si occupa della connettività del client. Il NSMgr e il registry si occupano di Service Discovery e configurazione della rete.
Vengono di seguito elencati i passaggi per l'instaurazione della connessione:

- NSM intercetta la richiesta di connessione del NSC al vl3-service
- il NSMgr del nodo trova il NSE che fornisce quel servizio
- NSM inietta dinamicamente una interfaccia kernel nel namespace del NSC, configura il routing e l'indirizzo IP (privato) e connette il Pod alla rete vl3 in cui ci sarà anche un altro NSC

7.1 K3s

La tecnologia Kubernetes negli ultimi anni si è affermata come standard de facto per la gestione di infrastrutture containerizzate. Tuttavia, la sua complessità, unita ai suoi requisiti di risorse elevati, ha portato alla nascita di soluzioni più leggere e orientate ad ambienti con risorse più limitate. ***K3s*** è tra le più popolari tra queste soluzioni, è un progetto open source sviluppato da Rancher Labs.

K3s è una distribuzione leggera di Kubernetes, progettata per essere semplice da installare, ridotta in termini di footprint (risorse di sistema necessarie per funzionare) e ottimizzata per scenari edge, embedded e ambienti a bassa potenza computazionale [18].

Il nome "K3s" gioca proprio su questa leggerezza: Kubernetes è una parola di 10 lettere riformulata come K8s, quindi il progetto di Rancher Labs, che richiede la metà della memoria ram, viene riformulato con la metà delle parole, ovvero K3s [18].

Le principali caratteristiche che distinguono K3s rispetto a una distribuzione tradizionale di Kubernetes includono [18]:

- Binary monolitico: K3s è distribuito come un singolo binario eseguibile, di dimensioni inferiori ai 100 MB
- Dipendenze minime: K3s elimina o sostituisce diverse dipendenze standard di Kubernetes, per esempio sostituisce etcd con SQLite
- Supporto integrato per containerd: il runtime container di default è containerd, più leggero e moderno rispetto a Docker.
- Ottimizzazione per il cloud e l'edge: K3s è pensato per funzionare anche su dispositivi a bassa potenza, come Raspberry Pi, o su ambienti virtualizzati leggeri (ad esempio VPS o VM con pochi core e Ram limitata).
- Supporto per modalità agent/server: K3s implementa un'architettura semplificata con un nodo server (che funge da Control Plane) e più nodi agent, facilitando il deployment distribuito.

Un nodo server è definito come un host che esegue il comando "**k3s server**", con Control Plane e componenti datastore gestiti da K3s. Un nodo agent è definito come un host che esegue il comando "**k3s agent**", senza nessun Control Plane o componenti datastore. Sia server che agent eseguono il kubelet, il container runtime e il CNI. K3s usa Flannel come CNI, che è già incluso nell'installazione. Rimane la possibilità di disabilitarlo e usarne un altro [18].

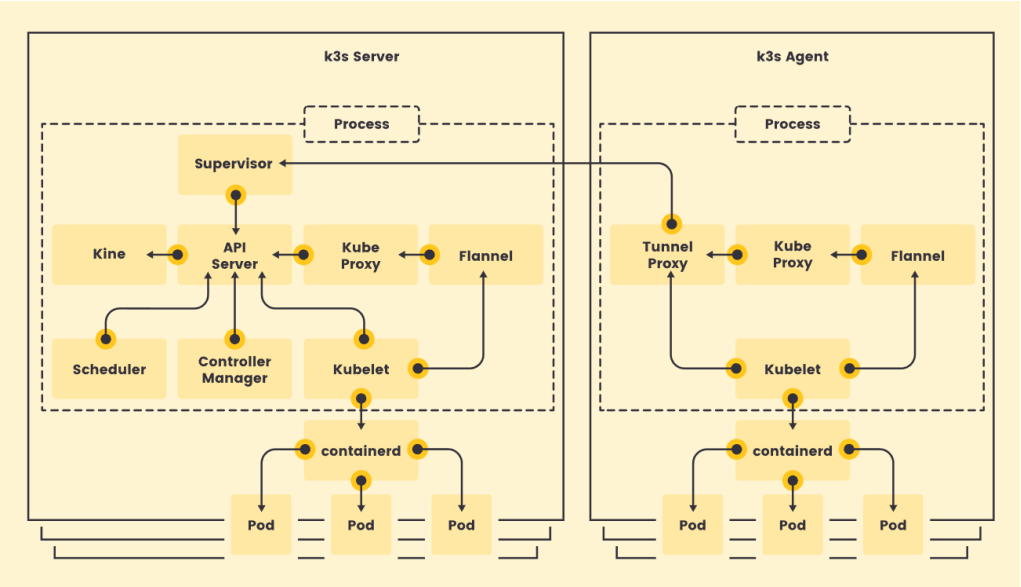

Figura 7.1: Architettura di K3s [https://docs.k3s.io/assets/images/how-it-works-k3s-revised-9c025ef482404bca2e53a89a0ba7a3c5.svg]

In Figura 7.1 si può vedere un confronto tra architettura di un server e un agent. Entrambe hanno componenti comuni come il kubelet, il kube-proxy e il CNI (Flannel). Il nodo server contiene anche il Control Plane.

7.2 Consumo risorse dei Control Plane di Istio e NSM

È utile confrontare l'utilizzo di memoria da parte di Istio e NSM al fine di valutarne l'impatto sulle prestazioni complessive del cluster.
I dati di seguito riportati sono stati raccolti con il comando "kubectl top pods" specificando il namespace su cui risiedono i Pod del Control Plane.
NSM fornisce un'identità ai suoi workload utilizzando Spire, mentre Istio utilizza un componente integrato chiamato Citadel. Per questo motivo, per Istio è stato rilevato solo il consumo di memoria del Control Plane (istiod), mentre per NSM è stato anche rilevato il consumo di memoria di Spire.
È da precisare che, opzionalmente, è possibile utilizzare Spire anche per Istio, questa configurazione è spesso preferita poiché Spire si adatta meglio ad architetture Zero Trust.

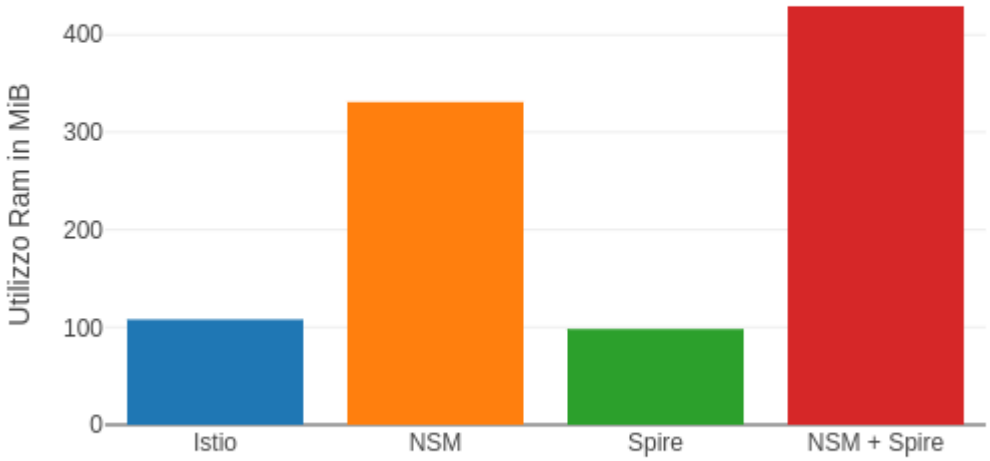

Figura 7.2: Consumo di risorse del Control Panel di Istio e NSM

Dal grafico si evince che il Control Plane di Istio ha un consumo nettamente inferiore di memoria rispetto a quello di NSM. Il Control Plane di NSM usufruisce di Spire, sono state quindi sommate le due colonne per ricavare l'utilizzo complessivo di memoria.

7.3 Tempo di avvio di ogni configurazione

In un cluster i Pod vengono creati, distrutti e riavviati di frequente. È quindi utile misurare il tempo di avvio delle quattro configurazioni prese in esame, ovvero il tempo che ci mettono i Pod ad arrivare in stato "Running". La misurazione è stata effettuata con semplici script Bash, in seguito è riportato un esempio.

```
1 #!/bin/bash
2 kubectl create namespace test
3 kubectl label namespace test istio-injection=enabled
4 START_TIME=$(date +%s)
5 kubectl apply -f client.yaml -n test
6 kubectl apply -f server.yaml -n test
7 kubectl wait --for=condition=ready pod -l app=debian-client -n test --timeout=2m
8 kubectl wait --for=condition=ready pod -l app=debian-server -n test --timeout=2m
9 END_TIME=$(date +%s)
10 echo "Startup time: $(($END_TIME - $START_TIME)) secondi"
```

Figura 7.3: Script usato per misurare il tempo di avvio della configurazione Istio

Come si può vedere in Figura 7.3, è stato usato il comando "kubectl wait", che mette in sospensione lo script fino a che il Pod a lui indicato non entra in stato "Ready".

In seguito vengono riportati i dati ottenuti dalle rilevazioni.

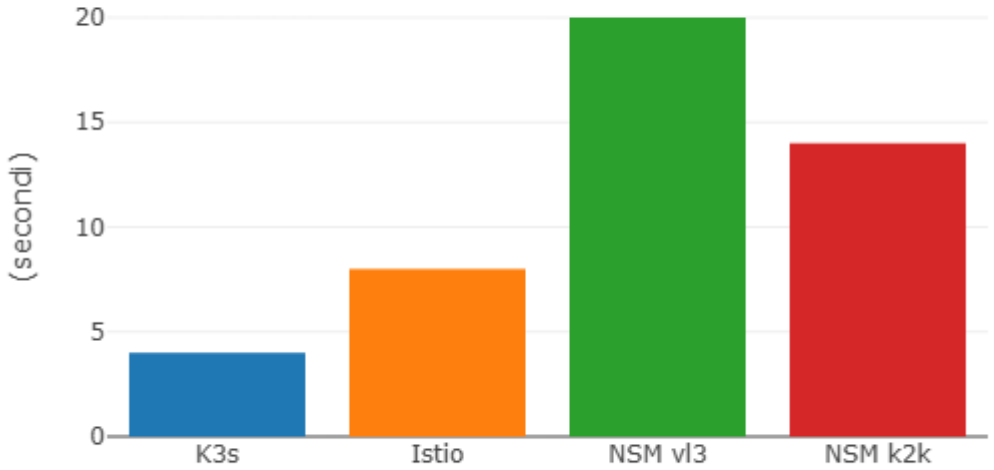

Figura 7.4: Tempo di avvio delle configurazioni K3s, Istio, NSM vl3 e NSM k2k

La configurazione vl3 di NSM è risultata la più lenta. La configurazione con Istio è risultata più veloce di entrambe quelle con NSM, ma ha comunque impiegato un tempo circa doppio rispetto alla configurazione con K3s. Bisogna considerare che Istio crea, per ogni Pod, un container con un sidecar proxy Envoy, perciò in questo caso sono stati creati il doppio dei container.

7.4 Test richieste HTTP

Spesso ai Pod Kubernetes viene assegnato il compito di fornire ai client dei servizi web, perciò devono essere in grado di rispondere velocemente ad una vasta quantità di richieste. Questa velocità, insieme al consumo di risorse, è stata valutata con il seguente test.
Per ogni configurazione del cluster è stato configurato un Pod che, tramite Nginx, serve una pagina web. L'altro Pod ha in esecuzione un'istanza di Linux Debian nella versione bullseye-slim. È stato usato il tool **wrk** per generare una elevata quantità di traffico. Quattro thread hanno tenuto aperte 100 connessioni verso il server Nginx per 30 secondi.

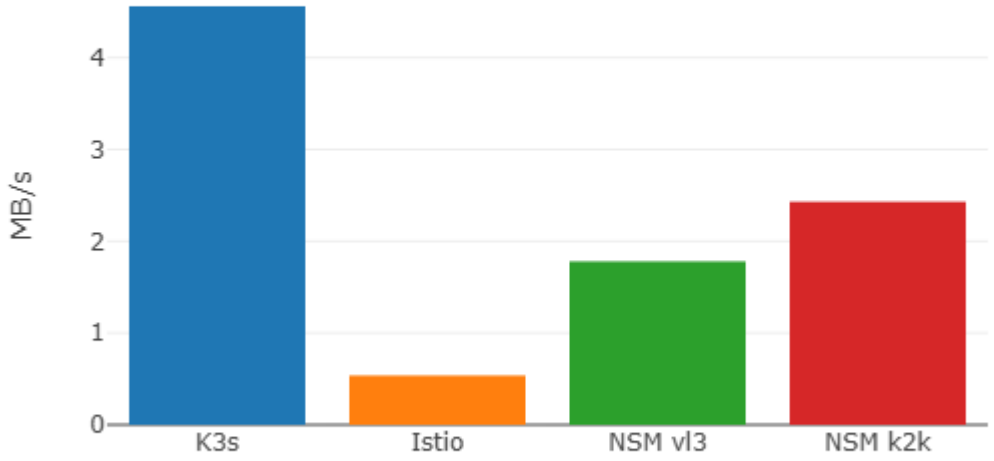

Figura 7.5: Velocità di risposta per ogni configurazione

In Figura 7.5 vengono riportate le velocità di trasferimento rilevate durante i test, per ogni configurazione. Il throughput di Istio è stato il più basso, si può supporre che ciò sia stato dovuto alla procedura di ritrasmissione delle risposte HTTP perse che solo Istio implementa.

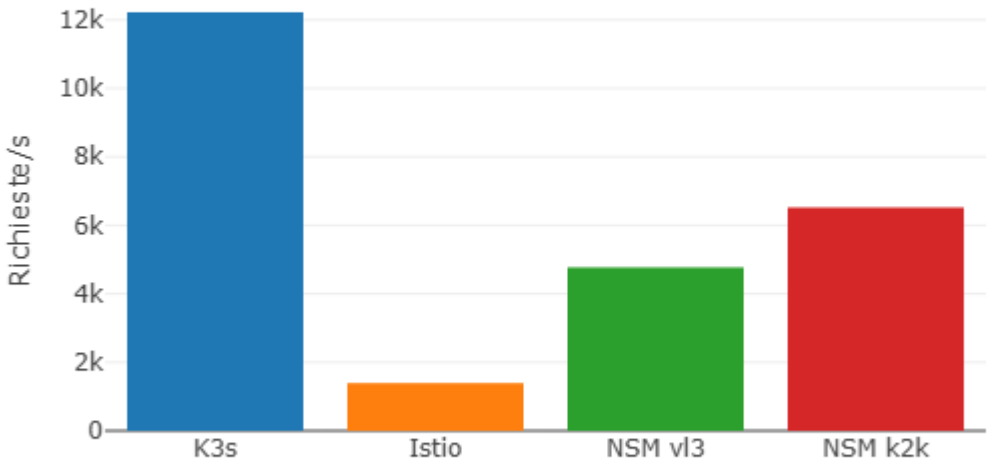

Figura 7.6: Richieste a cui viene data risposta, al secondo, per ogni configurazione

In Figura 7.6 viene riportato, per ogni configurazione, il numero di richieste a cui viene data una risposta con successo, al secondo. Le due figure sopra riportate mostrano un ampio divario di prestazioni a favore della configurazione K3s. La configurazione di Istio ha ottenuto il risultato peggiore, con circa 0.5 MB di throughput al secondo e meno di duemila richieste gestite al secondo.

È utile anche visualizzare il picco di consumo di risorse (CPU e Ram) che i Pod di queste configurazioni hanno richiesto. Attraverso Grafana si può visualizzare quanti core sta utilizzando un particolare namespace (tutti i suoi Pod). Quindi, per ogni configurazione di K3s, viene riportato l'utilizzo rilevato della CPU da parte dei namespace utilizzati per i test.

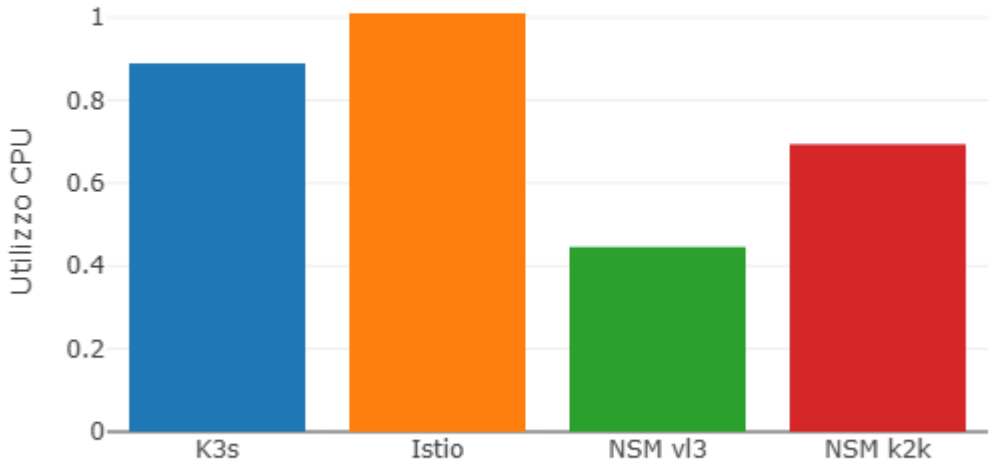

Figura 7.7: Picco di utilizzo CPU per ogni configurazione durante il test

Per ogni configurazione di K3s, viene riportato l'utilizzo rilevato di Ram da parte dei namespace utilizzati per i test:

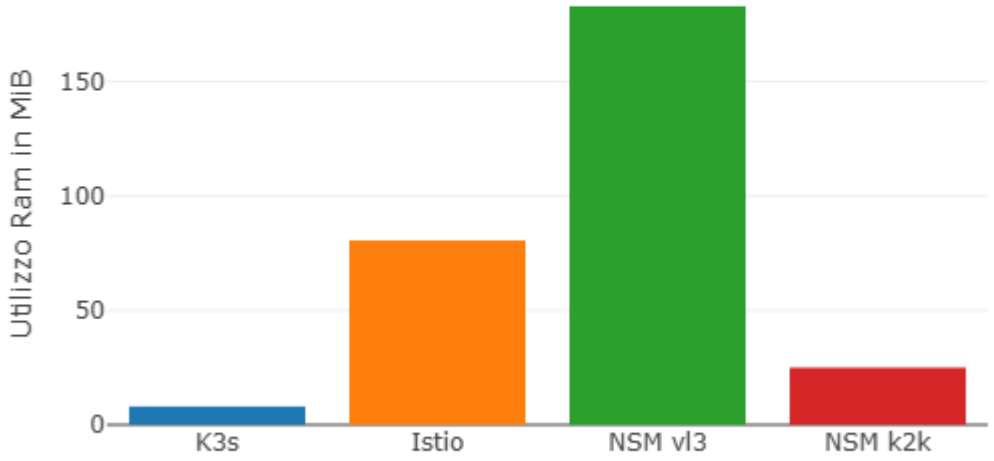

Figura 7.8: Picco di utilizzo Ram per ogni configurazione durante il test

In conclusione, nella Tabella 7.1 vengono riassunti i risultati ottenuti.

	Migliore risultato	Peggiore risultato
Throughput	K3s	Istio
Richieste al secondo	K3s	Istio
Utilizzo CPU	NSM vl3	Istio
Utilizzo Ram	K3s	NSM vl3

Tabella 7.1: Risultati ottenuti

Dai test si evince che nelle richieste HTTP tra una coppia di Pod Istio è il meno performante e quello che utilizza di più la CPU. K3s in configurazione standard rimane il miglior compromesso per questo tipo di comunicazione. È da notare che la configurazione k2k di NSM offre buone prestazioni a fronte di un basso consumo di risorse di CPU e Ram.

7.5 Test richieste TCP

Con questo test è stato valutato il throughput in una connessione TCP tra due Pod, che è il protocollo più sfruttato nelle comunicazioni tra Pod. È stato usato **iperf3**, un tool per la misurazione delle prestazioni di rete che consente di rilevare la larghezza di banda e la latenza nella comunicazione tra due sistemi.
Per ogni configurazione, in entrambe i Pod usati per i test è in esecuzione un container con Debian nella versione bullseye-slim. Di seguito vengono riportati i dati rilevati delle prestazioni della rete.

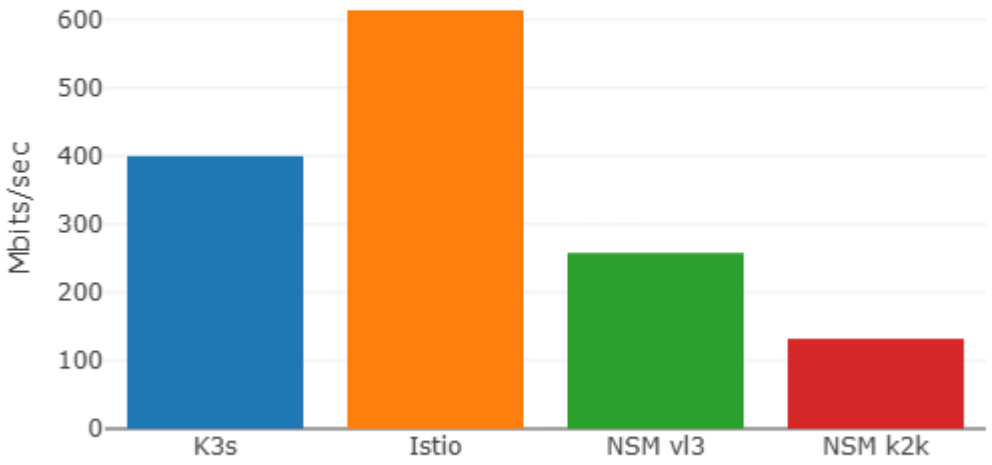

Figura 7.9: Prestazione della rete per ogni configurazione

Vengono poi riportati il numero di ritrasmissioni che è stato necessario fare, per ogni configurazione.

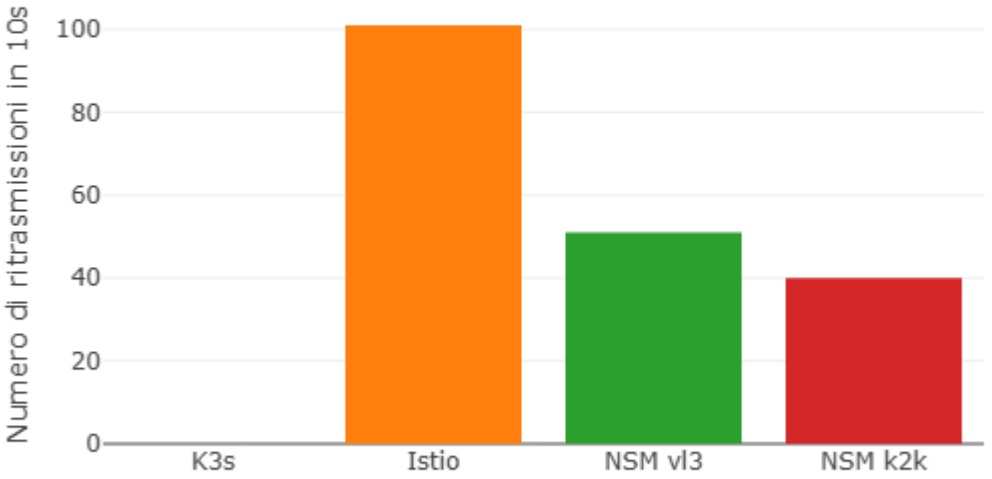

Figura 7.10: numero di ritrasmissioni effettuate da ogni configurazione durante il test

Da Figura 7.10 si può notare che la configurazione K3s non ha fatto ritrasmissioni, mentre le altre sì. Ciò potrebbe essere dovuto al fatto che la comunicazione tra due Pod connessi con K3s è molto semplice: si tratta di una connessione tra due socket di una stessa sottorete. Nel caso di Istio e NSM, invece, i Pod non comunicano direttamente mediante il CNI ma il traffico passa attraverso Envoy o Forwarder. La complessità della comunicazione è quindi molto più elevata e la perdita di pacchetti è più probabile.
È da precisare che K3s, Istio e NSM usufruiscono del meccanismo di ritrasmissione di TCP. Solo Istio implementa anche delle proprie modalità di ritrasmissione.

È utile anche visualizzare il picco di consumo di risorse (CPU e Ram) che i Pod di queste configurazioni hanno richiesto. Attraverso Grafana si può visualizzare quanti core sta utilizzando un particolare namespace (tutti i suoi Pod). Quindi, per ogni configurazione di K3s, viene riportato l'utilizzo rilevato della CPU da parte dei namespace utilizzati per i test:

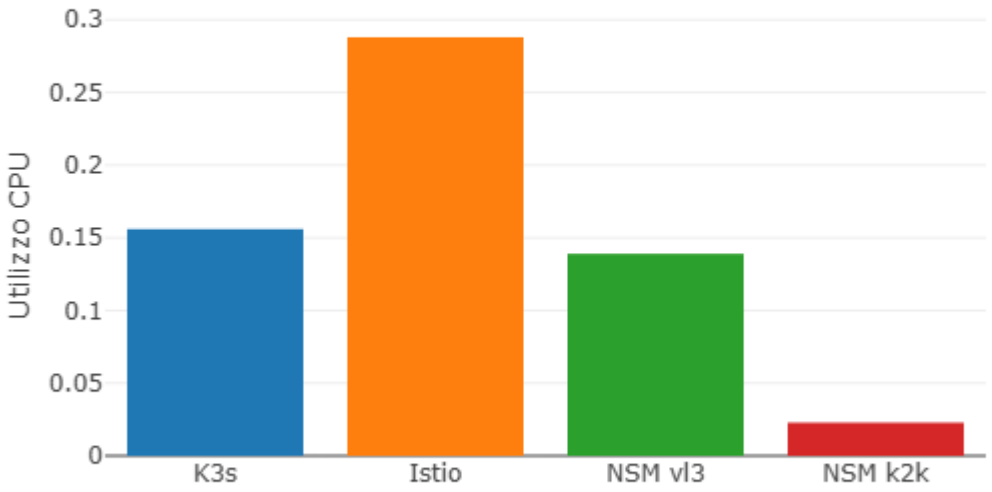

Figura 7.11: Picco di utilizzo CPU per ogni configurazione durante il test

Per ogni configurazione di K3s, viene riportato l'utilizzo rilevato della Ram da parte dei namespace utilizzati per i test:

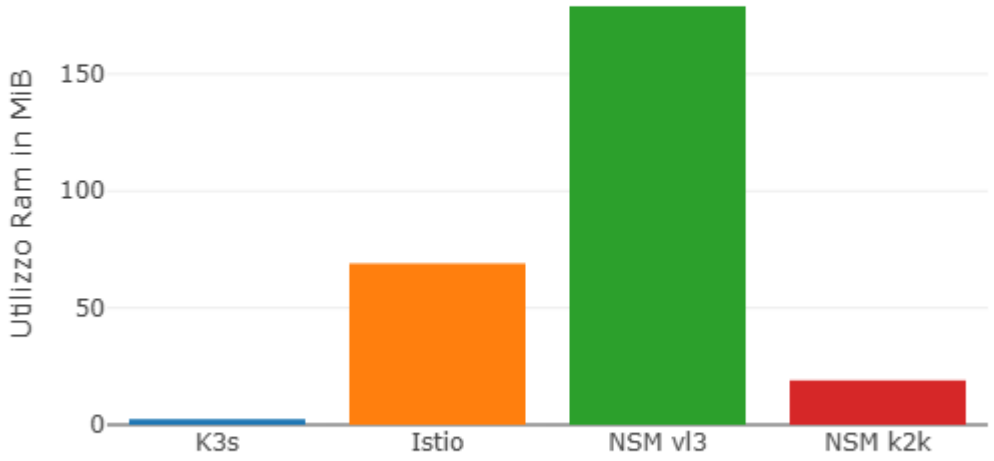

Figura 7.12: Picco di utilizzo Ram per ogni configurazione durante il test

In conclusione, in Tabella 7.2 vengono riassunti i risultati ottenuti.

	Migliore risultato	Peggiore risultato
Throughput	Istio	NSM k2k
Numero di ritrasmissioni	K3s	Istio
Utilizzo CPU	NSM k2k	Istio
Utilizzo Ram	K3s	NSM vl3

Tabella 7.2: Risultati ottenuti

Dai dati raccolti si evince che, per una connessione TCP, la configurazione di Istio è un buon compromesso tra prestazioni e consumo di risorse. La differenza di velocità di trasmissione con K3s standard non è però così elevata e, in alcuni casi, potrebbe non giustificare la complessità introdotta dal suo utilizzo e il suo consumo di risorse. Le prestazioni rilevate per entrambe le configurazioni di NSM sono inferiori. Da notare inoltre che la configurazione vl3 di NSM ha comportato un elevato consumo di memoria in confronto alle altre.

7.6 Test richieste gRPC

Con il seguente test si è voluto misurare la latenza tra chiamate gRPC tra un client ed un server che risiedono in due Pod diversi. Sono stati usati due semplici programmi scritti in Go per far scambiare un messaggio di saluto da server a client. In entrambe i Pod usati per i test è in esecuzione un container con Debian

nella versione bullseye-slim. Nel container server c'è un demone Go in ascolto. È stato usato uno strumento chiamato **ghz** per eseguire il programma client ripetutamente al fine di fare mille interrogazioni al server e misurare la latenza. Di seguito vengono riportati i dati rilevati su latenza per il 99% delle richieste e sulle richieste completate al secondo.

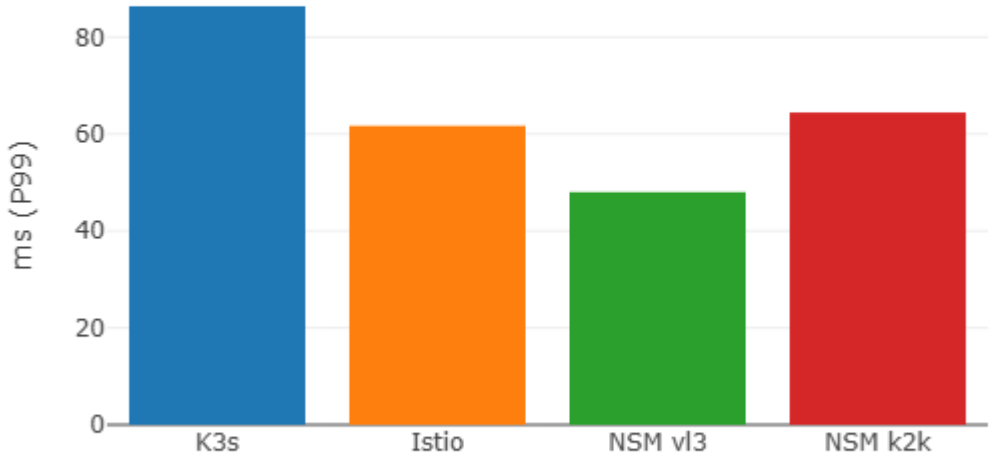

Figura 7.13: Latenza per ogni configurazione durante il test

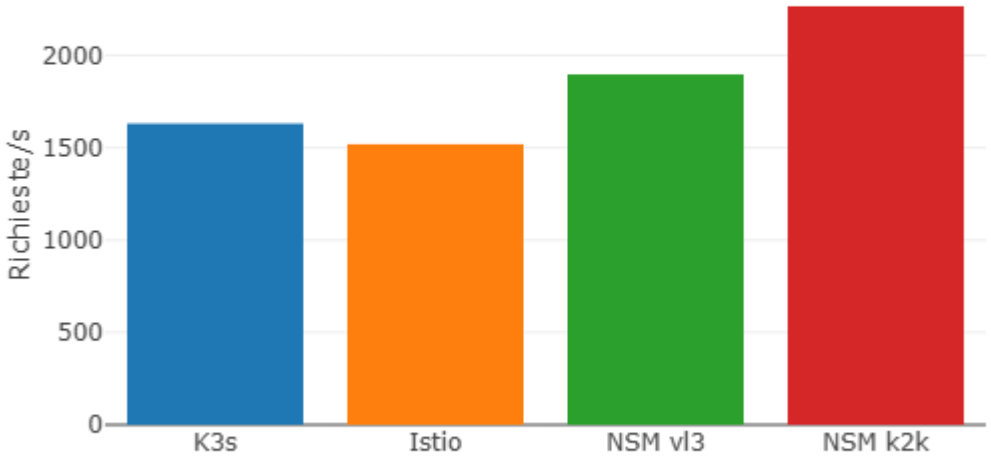

Figura 14: Richieste completate al secondo, per ogni configurazione

Sono stati anche raccolti dati sulla varianza della latenza nelle risposte, per ogni configurazione.

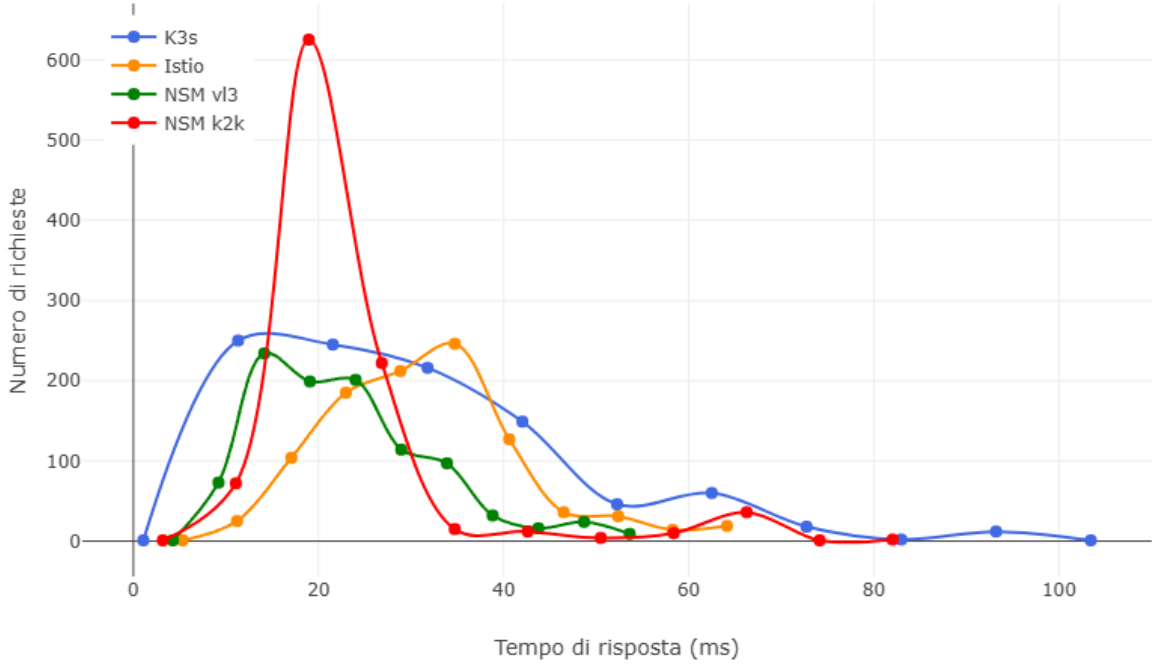

Figura 7.15: Numero di richieste a cui è stata data risposta con una data latenza, espressa nell'asse delle ascisse

La Figura 7.15 indica il numero di richieste a cui è stata data risposta con una determinata latenza, espressa nell'asse delle ascisse. Si può notare che la

configurazione k2k di NSM ha gestito la maggior parte delle richieste con una latenza circa di 20 ms. La configurazione vl3 di NSM è quella che ha avuto meno varianza nella latenza delle risposte, infatti le sue latenze variano in un intervallo di soli 55 ms circa. La configurazione K3s, al contrario, ha comportato una elevata varianza nella latenza delle risposte, che si trovano in un intervallo di circa 100 ms.

È utile anche visualizzare il picco di consumo di risorse (CPU e Ram) che i Pod di queste configurazioni hanno richiesto. Attraverso Grafana si può visualizzare quanti core sta utilizzando un particolare namespace (tutti i suoi Pod). Quindi, per ogni configurazione di K3s, viene riportato l'utilizzo rilevato della CPU da parte dei namespace utilizzati per i test.

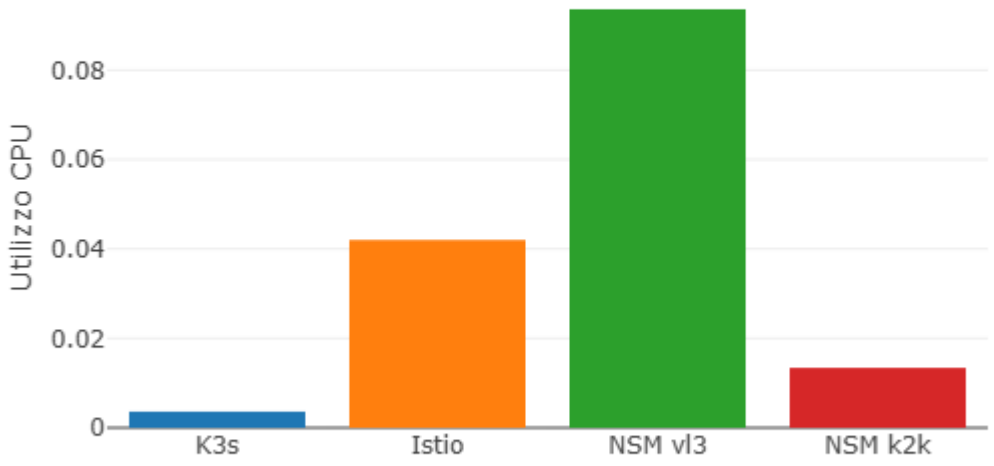

Figura 7.16: Picco di utilizzo CPU per ogni configurazione durante il test

Per ogni configurazione di K3s, viene riportato l'utilizzo rilevato della Ram da parte dei namespace utilizzati per i test.

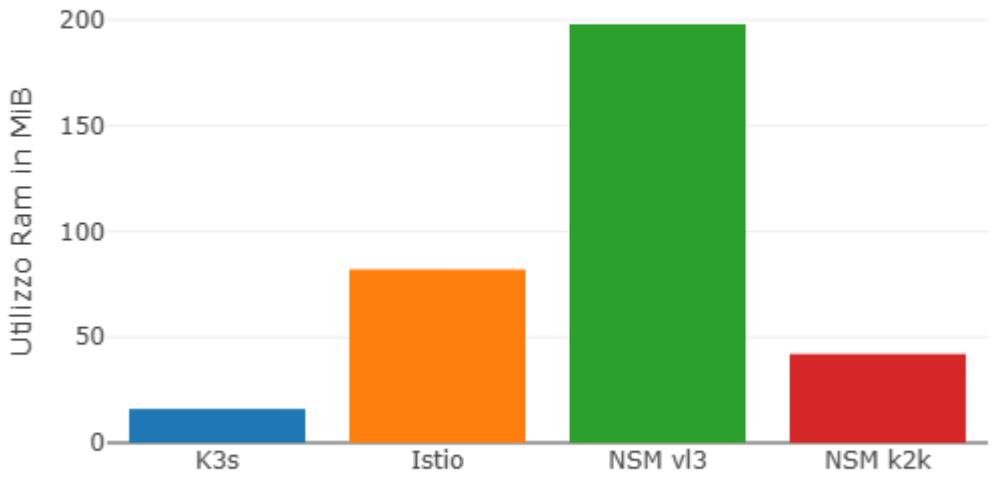

Figura 7.17: Picco di utilizzo Ram per ogni configurazione durante il test

Come si può vedere da Figura 7.16 e 7.17 la configurazione vl3 di NSM ha utilizzato molte più risorse rispetto alle altre.

Per concludere, la Tabella 7.3 evidenzia i migliori risultati sperimentali ottenuti in questa sezione.

	Migliore risultato	Peggiore risultato
Latenza	NSM vl3	K3s
Richieste al secondo	NSM k2k	Istio
Utilizzo CPU	K3s	NSM vl3
Utilizzo Memoria	K3s	NSM vl3
Varianza della latenza nelle risposte	NSM vl3	K3s

Tabella 7.3: Risultati ottenuti

Dai dati raccolti si evince che, per effettuare chiamate gRPC tra due Pod, la configurazione k2k di NSM risulta un buon compromesso tra utilizzo di risorse e prestazioni. La configurazione K3s semplice consuma pochissime risorse ma introduce un'ampia latenza nelle risposte. La configurazione vl3 di NSM ha ottenuto il risultato migliore per la latenza nelle risposte ma ha comportato un utilizzo di risorse molto più elevato di quello delle altre configurazioni.

Nei sottocapitoli 7.4, 7.5 e 7.6 le comunicazioni con HTTP, TCP e gRPC sono

state oggetto di test continuativi, per creare un grande volume di traffico. La velocità di trasmissione di Istio è stata superiore a quella di NSM per TCP, mentre è stata simile a quella di NSM per gRPC. Nel caso di richieste HTTP il throughput di Istio è stato il più basso. Si può supporre che ciò sia stato dovuto alla procedura di ritrasmissione delle risposte HTTP perse. Istio implementa questa funzionalità, NSM no.
Le prestazioni rilevate dei Pod nella configurazione K3s semplice sono state simili a quelle di NSM e Istio, mentre il consumo di risorse (in particolare di memoria) è stato nettamente inferiore. È da notare inoltre che la configurazione vl3 di NSM ha sempre comportato un elevato utilizzo di memoria, che può essere imputato alla tecnologia VPP utilizzata per processare gruppi di pacchetti simultaneamente.

7.7 Test traffico variabile

Obiettivo di questo test è stato sottoporre le quattro configurazioni a diversi tipi di carico per valutarne le prestazioni. I tipi di carico utilizzati sono stati esponenziale, lineare e Gaussiano, al fine di simulare situazioni reali in cui le richieste dei client aumentano nel tempo, in modo diverso.
Per ogni configurazione, un Pod contiene un container che serve una pagina web statica tramite Nginx. In un altro Pod c'è un container con in esecuzione un'istanza di Linux Debian in versione bullseye-slim, da cui sono partite le richieste per il servizio.
Per questi test Istio è stato configurato per una precisa gestione del traffico, al fine di evitare un eccessivo consumo di risorse.

```yaml
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: my-static-site
spec:
  hosts:
  - "*"
  gateways:
  - my-static-site-gateway
  http:
  - match:
    - uri:
        prefix: /
    route:
    - destination:
        host: my-static-site
        port:
          number: 80
  - timeout: 5s
    route:
    - destination:
        host: my-static-site.default.svc.cluster.local
```

Figura 7.18: File di configurazione VirtualService usato per il test

Il VirtualService in Figura 7.18 stabilisce che se un Pod impiega più di cinque secondi per rispondere allora la richiesta fallisce e verrà restituito un errore 504.

```yaml
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: my-static-site
spec:
  host: my-static-site.default.svc.cluster.local
  trafficPolicy:
    loadBalancer:
      simple: ROUND_ROBIN
    connectionPool:
      tcp:
        maxConnections: 100
      http:
        http2MaxRequests: 1000
        maxRequestsPerConnection: 10
    outlierDetection:
      consecutive5xxErrors: 7
      interval: 10s
      baseEjectionTime: 30s
```

Figura 7.19: File di configurazione DestinationRule usato per il test

La DestinationRule in Figura 7.19 stabilisce che in ogni proxy potranno avvenire al massimo 100 connessioni TCP contemporaneamente e al massimo 1000 richieste HTTP2 concorrenti per ogni connessione. Ogni connessione verrà chiusa e riaperta dopo 10 richieste HTTP/1.1. Infine, ogni 10 secondi, verrà fatto un controllo per cui se un Pod ha fallito più di 7 richieste allora non gli verrà più assegnato carico per 30 secondi.

Il carico è stato creato con uno strumento chiamato **K6** che permette di simulare tanti utenti virtuali che fanno richieste HTTP, in modo da verificare la capacità e la resilienza di un sistema. Per ogni configurazione sono stati raccolti dati sulla variazione nel tempo della latenza delle risposte, sulla latenza del servizio per il 95% delle richieste e sul numero di richieste effettuate e fallite al servizio.

Nel primo test il carico di richieste al sistema è stato minimo durante i primi secondi della rilevazione, per poi aumentare istantaneamente e vertiginosamente. Di seguito viene riportato lo script usato.

```
1 import http from 'k6/http';
2 import { sleep } from 'k6';
3
4 export let options = {
5   stages: [
6     { duration: '10s', target: 150 },
7     { duration: '0', target: 10000 },
8     { duration: '1m', target: 10000 },
9     { duration: '10s', target: 100 },
10   ],
11 };
12
13 export default function () {
14   http.get('http://my-static-site.default.svc.cluster.local:80/');
15   sleep(1);
16 }
```

Figura 7.20: Script K6 usato per il test

Come si può vedere da Figura 7.20, dopo 10 secondi arrivano in modo immediato diecimila utenti virtuali. Questi fanno, per un minuto, richieste continue al sistema ad un intervallo di un secondo l'una dall'altra. Infine, nell'arco di dieci secondi, gli utenti virtuali calano a 100.

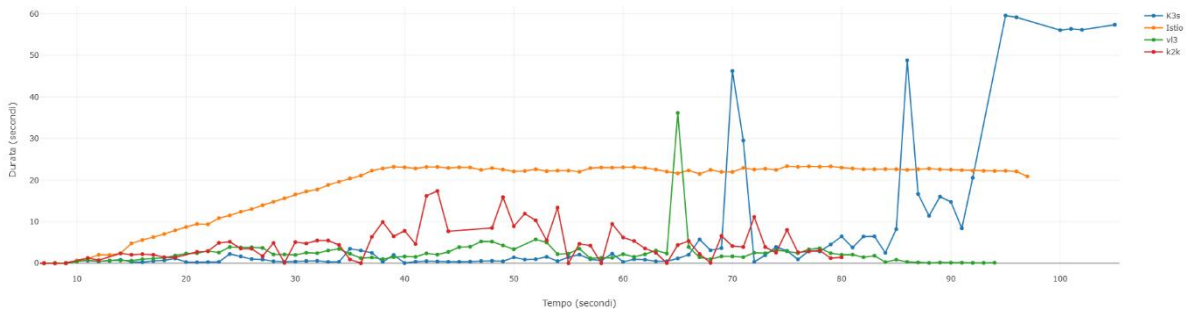

Figura 7.21: Latenza per ogni richiesta, per ogni configurazione, nel tempo

Figura 7.21 mostra nell'asse del tempo, quanto tempo è stato necessario per rispondere ad una richiesta. Durante il test, circa al decimo secondo della rilevazione il carico è aumentato esponenzialmente, passando da 150 a 10000 utenti virtuali. Dai dati rilevati si può notare che, con l'aumentare del carico, il tempo di risposta di Istio è aumentato ma è poi rimasto costante per tutta la rilevazione. Le configurazioni di NSM hanno entrambe performato meglio di Istio e K3s, ma in modo meno costante di Istio e con alcuni picchi di latenza molto elevati. Verso la fine della rilevazione K3s ha evidentemente terminato le risorse a lui disponibili, infatti la latenza è aumentata vertiginosamente.

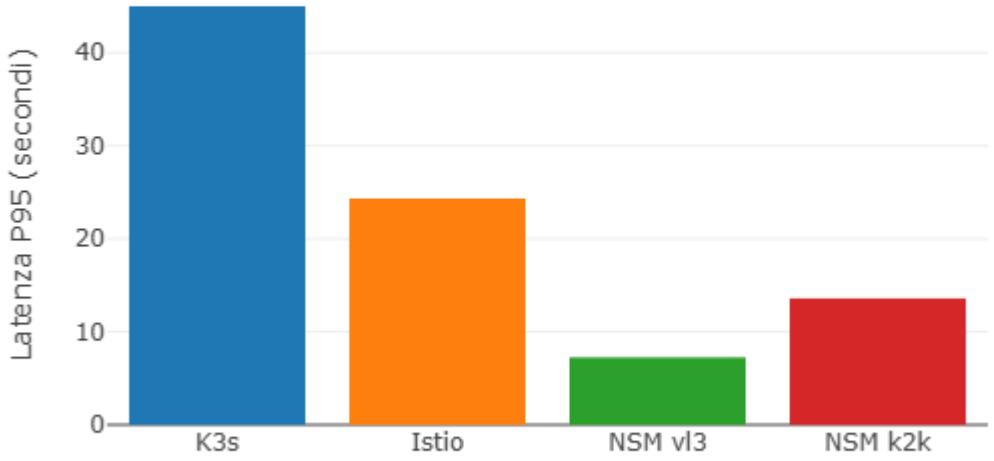

Figura 7.22: Latenza per il 95% delle richieste, per ogni configurazione, in secondi

La Figura 7.22 mostra la latenza generale in tutto il test, per ogni configurazione, per il 95% delle richieste. Si può vedere che la latenza rilevata per Istio è nettamente inferiore a quella di K3s ma non inferiore a quella di NSM.

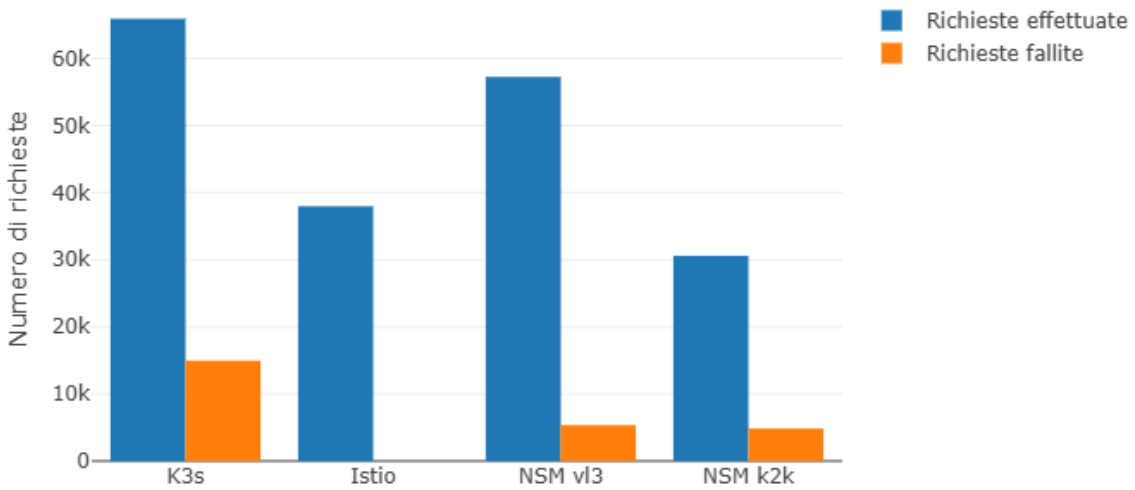

Figura 7.23: Numero di richieste effettuate e fallite, per ogni configurazione

Dai dati esposti in Figura 7.21, Figura 7.22 e Figura 7.23 si può vedere che la configurazione K3s semplice è quella che ha gestito più richieste ma ha introdotto picchi di latenza ben superiori a quella delle altre configurazioni. Istio ha mantenuto la latenza delle risposte costante nelle fasi di alto carico, che è rimasta non eccessivamente elevata. È da notare il fatto che Istio non ha fatto fallire nessuna richiesta, in quanto i proxy Envoy si occupano anche di ritrasmissione dei pacchetti persi.
La configurazione vl3 di NSM ha performato meglio delle altre, in tutte e tre le rilevazioni.

Nel secondo test il carico è stato fatto aumentare in modo lineare. Lo script usato è molto simile a quello in Figura 7.16.

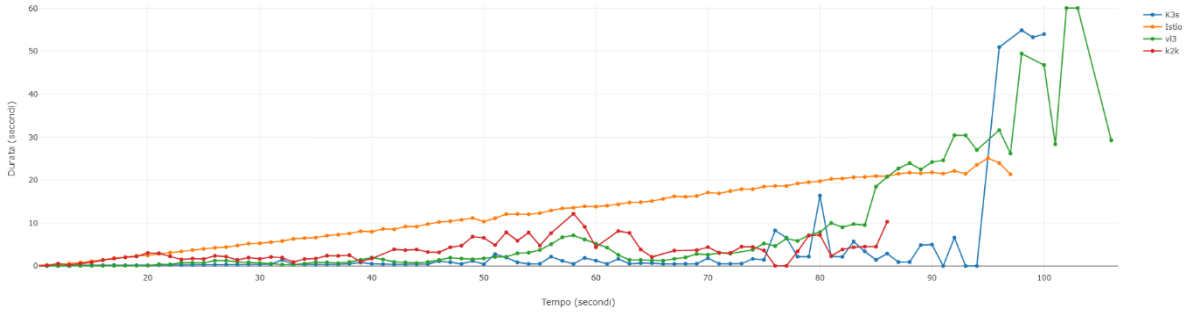

Figura 7.24: Latenza per ogni richiesta, per ogni configurazione, nel tempo

In Figura 7.24 si può notare che Istio, con l'aumentare del carico, ha aumentato gradualmente il tempo per rispondere ad ogni richiesta. Per la configurazione vl3 di NSM la latenza è minore per buona parte della rilevazione ma, verso la fine, i tempi sono aumentati drasticamente. La configurazione k2k di NSM, invece, ha mantenuto una latenza bassa e senza particolari picchi positivi ed ha esaurito le richieste a cui rispondere prima delle altre configurazioni.

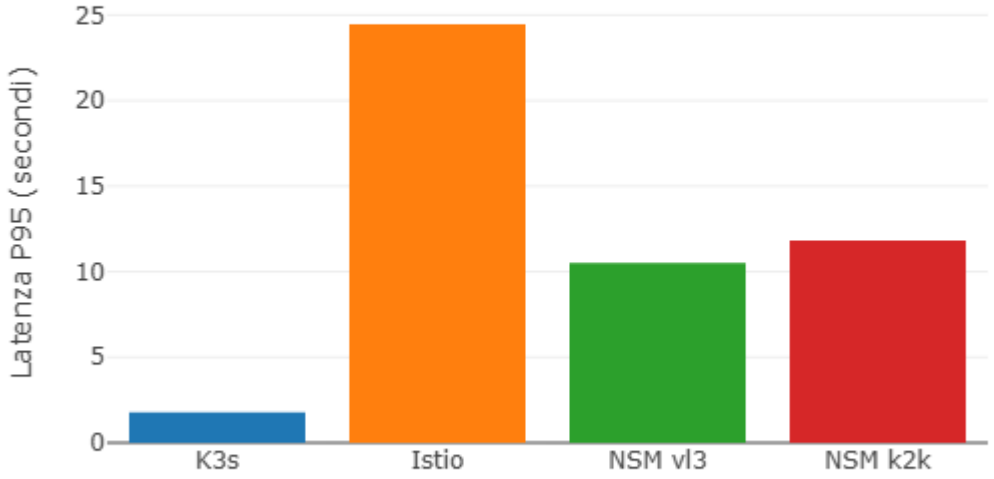

Figura 7.25: Latenza per il 95% delle richieste, per ogni configurazione, in secondi

Figura 7.25 mostra la latenza generale in tutto il test, per il 95% delle richieste. Si può notare che la latenza introdotta da Istio è molto elevata, soprattutto in confronto a K3s.

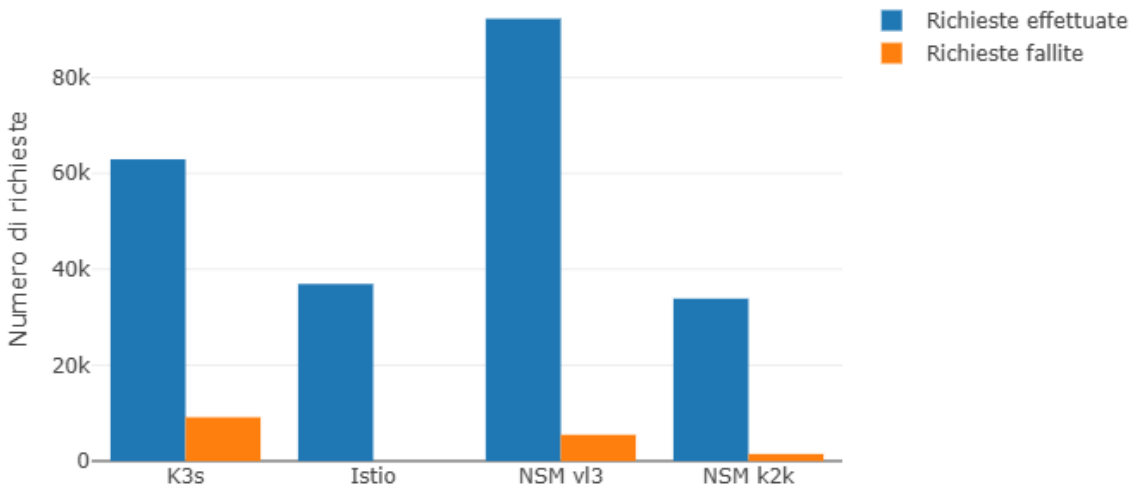

Figura 7.26: Numero di richieste effettuate e fallite, per ogni configurazione

In Figura 7.26 si può notare che la configurazione vl3 di NSM ha gestito il maggior numero di richieste e con un basso numero di richieste fallite. Istio ritrasmette i pacchetti persi perciò non ha fatto fallire richieste, ma ha gestito (come la configurazione k2k) un minor numero di richieste rispetto a K3s.
Anche in questo test Istio si è distinto per non aver provocato un aumento della latenza all'aumentare delle richieste. Le sue prestazioni sono state però non all'altezza di quelle delle altre configurazioni. La configurazione vl3 è risultata un buon compromesso per latenza e numero di richieste gestite, anche se la latenza introdotta da K3s semplice è stata nettamente inferiore rispetto a quella delle altre configurazioni.

Nel terzo test il carico è stato incrementato e diminuito simulando l'andamento della curva di Gauss.

```
1 import http from 'k6/http';
2 import { sleep } from 'k6';
3
4 export let options = {
5     scenarios: {
6         gaussian_traffic: {
7             executor: 'ramping-arrival-rate',
8             startRate: 10, // RPS (Request Per Second) iniziale
9             timeUnit: '1s',
10            preAllocatedVUs: 200,
11            maxVUs: 10000,
12            stages: generateGaussianStages(80, 10000, 20),
13            exec: 'sendRequest',
14        },
15    },
16 };
17
18 // Funzione che genera degli stage (fasi) per approssimare la curva Gaussiana
19 function generateGaussianStages(totalDurationSec, maxRPS, steps) {
20     const stages = [];
21     const mean = totalDurationSec / 2;
22     const stddev = totalDurationSec / 6;
23     for (let i = 0; i <= steps; i++) {
24         const t = (i / steps) * totalDurationSec;
25         // Formula della curva Gaussiana:
26         const rps = Math.round(maxRPS * Math.exp(-Math.pow(t - mean, 2) / (2 * stddev * stddev)));
27         stages.push({ duration: `${totalDurationSec / steps}s`, target: rps });
28     }
29     return stages;
30 }
31
32 export function sendRequest() {
33     http.get('http://my-static-site.default.svc.cluster.local:80/');
34     sleep(1);
35 }
```

Figura 7.27: Script K6 utilizzato per il test

In Figura 7.27 viene riportato lo script utilizzato per il test, in cui viene campionata la curva di Gauss in modo tale da ricavare, per ogni punto, il numero di utenti virtuali nel tempo.

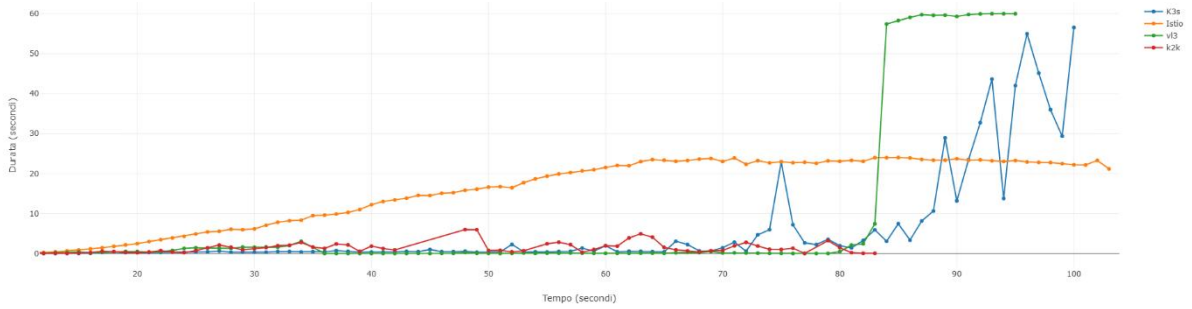

Figura 7.28: Latenza per ogni richiesta, per ogni configurazione, nel tempo

Durante il test il picco degli utenti virtuali è avvenuto circa al secondo 40 della rilevazione. In Figura 7.28 si può vedere che, anche per questo test, Istio ha mantenuto la latenza delle sue risposte costante nel tempo, anche se risulta molto più elevata di quella delle altre configurazioni. La configurazione k2k di NSM è stata la prima a terminare tutte le richieste a cui rispondere e non ha comportato particolari picchi positivi di latenza, a differenza delle configurazioni vl3 di NSM e K3s. Dopo circa 80 secondi dall'inizio del test si può vedere un aumento vertiginoso della latenza per la configurazione vl3 di NSM e anche per quella di K3s.

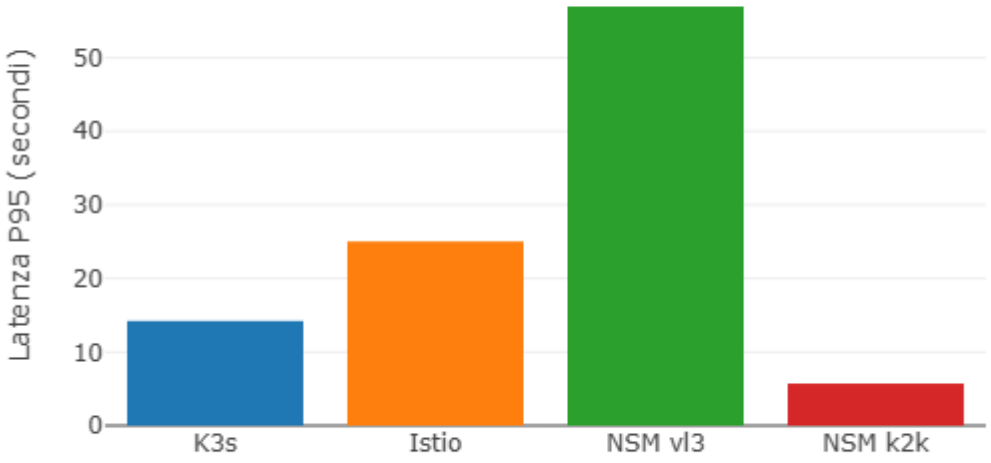

Figura 7.29: Latenza per il 95% delle richieste, per ogni configurazione, in secondi

La Figura 7.29 mostra la latenza generale in tutto il test, per ogni configurazione, per il 95% delle richieste. Si può notare che la configurazione vl3 di NSM ha introdotto una latenza nelle risposte ben superiore a quella delle altre

configurazioni, mentre la configurazione k2k di NSM ha avuto il risultato migliore.

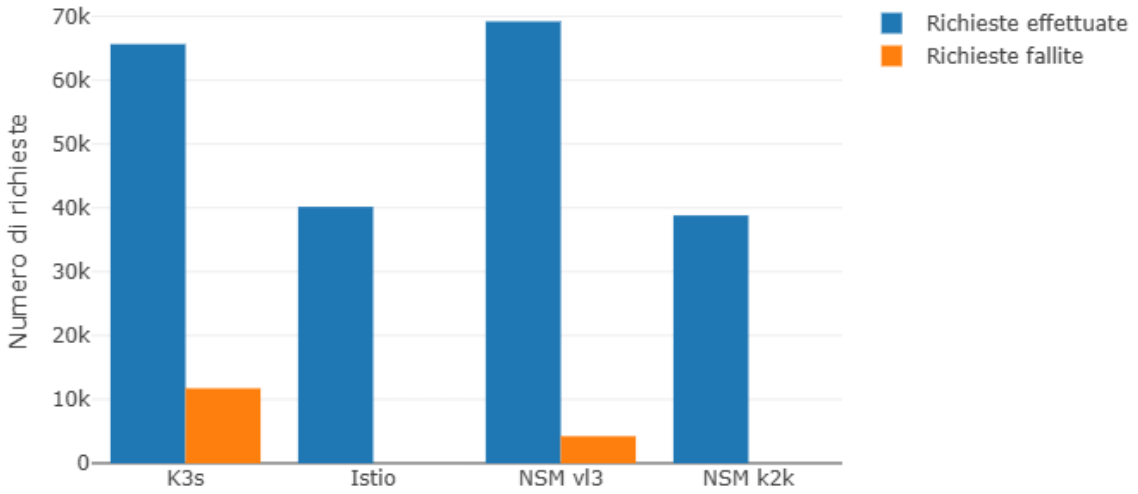

Figura 7.30: Numero di richieste effettuate e fallite, per ogni configurazione

In Figura 7.30 si può vedere che il numero di richieste gestite dalla configurazione vl3 di NSM risulta il più alto e con un basso numero di richieste fallite. Il numero di richieste gestite da Istio è stato inferiore, ma la ritrasmissione dei pacchetti non ha comportato alcuna richiesta fallita. È da notare che anche la configurazione k2k di NSM non ha fatto fallire nessuna richiesta, in generale è stata quella che ha avuto il miglior compromesso tra numero di richieste gestite e latenza introdotta.

In seguito, verranno tratte alcune conclusioni sui dati raccolti nei test svolti.

	Carico Esponenziale	Carico Lineare	Carico Gaussiano
Minore varianza di latenza	Istio	Istio	Istio
Minore latenza	NSM vl3	K3s	NSM k2k
Maggiori richieste effettuate	K3s	NSM vl3	NSM vl3
Maggiori richieste fallite	K3s	K3s	K3s
Minori richieste fallite	Istio	Istio	Istio/NSM k2k

Tabella 7.4: Migliori risultati per ogni tipo di test svolto

La configurazione con **Istio** è stata per tutti e tre i test quella che ha gestito meglio la variazione del carico ed è risultata la più **affidabile**. Infatti, non ha mai comportato picchi positivi o negativi di latenza, che è rimasta sempre costante oppure è aumentata gradualmente. Tutte le richieste hanno avuto risposta, grazie alla funzionalità di ritrasmissione dei pacchetti persi fornita dal proxy Envoy.
Le sue prestazioni sono risultate però inferiori a quelle delle altre configurazioni.

La configurazione **vl3** di NSM ha avuto in generale buone prestazioni. L'unica criticità si è manifestata nel caso di traffico con andamento Gaussiano, in cui le prestazioni sono peggiorate nel tempo, con un drastico aumento della latenza per le ultime richieste. Infine, il numero di richieste gestite è stato superiore a quello delle altre configurazioni, ma durante le rilevazioni ci sono stati diversi **picchi positivi di latenza**.

La configurazione **k2k** di NSM è stata il **miglior compromesso** tra prestazioni e gestione del carico crescente. Infatti, in tutti i test, l'andamento nel tempo della latenza alle risposte non è variato in modo significativo con l'aumentare del carico. Il numero di richieste gestite non è mai stato tanto inferiore rispetto a quello delle altre configurazioni e la latenza generale di ogni test è sempre stata discreta. Infine, questa configurazione ha terminato per prima di rispondere a tutte le richieste, in tutti e tre i test.

La configurazione **K3s** è stata quella dalle **prestazioni più altalenanti**. In ogni test ha gestito bene il carico crescente per la prima parte della rilevazione, per poi aumentare vertiginosamente la latenza per le risposte. Il numero di richieste gestite è stato alto rispetto alle altre configurazioni, ma a fronte di diverse richieste fallite.

I test con traffico crescente in modo esponenziale, lineare e Gaussiano sono stati fatti per simulare tre casi diversi di traffico a cui può essere soggetto un Pod che serve una applicazione web. Dai grafici risultanti si può notare che, in ogni configurazione, la latenza per le risposte aumenta all'aumentare del carico.
Le configurazioni k2k di NSM e Istio sono quelle che hanno gestito meglio le proprie risorse, infatti in tutti i test non si sono mai manifestati picchi positivi di latenza nelle risposte. Le configurazioni vl3 di NSM e K3s hanno invece comportato una **saturazione delle risorse** a loro disponibili, comportando in ogni test un aumento della latenza, per ogni richiesta.
In conclusione, la configurazione di Istio è stata la più affidabile e quella con meno varianza di latenza nelle risposte, nel tempo. Le sue prestazioni sono state però inferiori rispetto alle altre configurazioni.

7.8 Test simulazione fallimento del Data Plane

Obiettivo del seguente test è stato verificare cosa succede quando fallisce il Data Plane in una comunicazione tra Pod, prima nel caso di Istio e poi NSM.

Il primo test è stato effettuato tra due Pod con in esecuzione un'istanza di Debian in versione bullseye-slim. È stato avviato un test **iperf3** della durata di un minuto in cui dopo trenta secondi è stato fermato il container con in esecuzione il proxy Envoy.

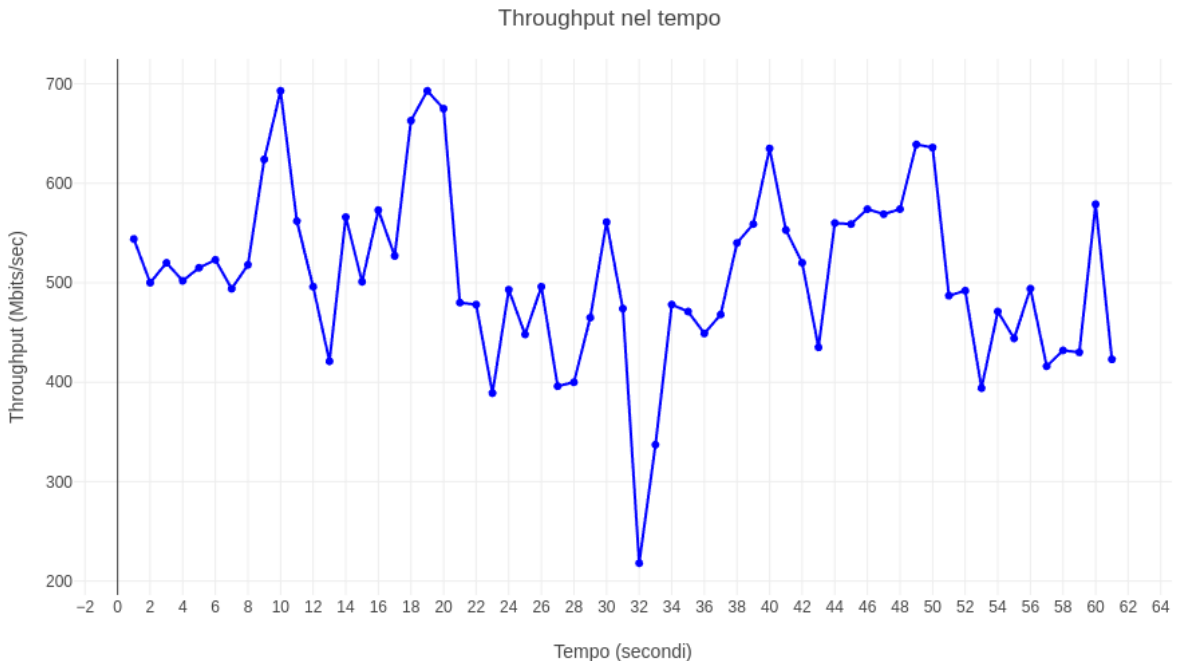

Figura 7.31: Throughput risultante dal test con iperf3, nel tempo

Come si può vedere da Figura 7.31, quando è stato fermato il Proxy Envoy il Control Plane di Istio lo ha immediatamente ricreato in meno di un secondo. Infatti, il test non si è fermato ma ha avuto solo un picco negativo di prestazioni.

Per quanto riguarda NSM, il test è stato fatto con una configurazione vl3. È stato simulato il fallimento del Data Plane VPP in uno dei NSE disabilitando tutte le sue interfacce memif. In seguito, viene riportato lo script usato.

```
1 #!/bin/bash
2 nses=$(kubectl get pods -l app=nse-vl3-vpp -n ns-vl3-config --template '{{range .items}}{{.metadata.name}} {{end}}')
3 NSE1=$(echo $nses | cut -d " " -f 1)
4 ifaces=$(kubectl exec -n ns-vl3-config $NSE1 -- vppctl show int | grep memif | awk '{print $1}' | tr '\n' ' ')
5 for if in $ifaces
6 do
7     kubectl exec -n ns-vl3-config $NSE1 -- vppctl set interface state $if down
8 done
```

Figura 7.32: Script usato nel test per disabilitare le interfacce memif in un NSE

Lo script in Figura 7.32 prende il nome preciso di un Pod NSE, poi esegue comandi all'interno del Pod per ricavare la lista delle sue interfacce memif e in seguito le disabilita tutte.

A questo punto è stato possibile verificare che la connettività tra i client e la connettività tra client e i restanti NSE è stata nuovamente instaurata. Il NSC, che monitora di continuo la connessione, si è accorto del malfunzionamento ed ha richiesto una nuova connessione. Il traffico è stato dirottato verso i rimanenti NSE. Ciò è possibile perché le interfacce create da NSM nei vari Pod sono ridondanti: esistono diversi percorsi che il traffico può prendere. Infine, è da sottolineare il fatto che nell'eventualità del fallimento di VPP in un NSE, vengono assegnati nuovi indirizzi IP ai client e ciò può comportare un malfunzionamento della logica applicativa. Non è perciò possibile effettuare un test di connessione continuativo con iperf3 con fallimento del Data Plane.

In conclusione, sia Istio che NSM sono resilienti al fallimento del Data Plane in quanto ripristinano le connessioni. NSM però introduce una difficoltà in quanto cambia gli indirizzi IP dei client. Questo problema con Istio non avviene poiché è il CNI di Kubernetes ad assegnare gli indirizzi IP ai Pod, non il proxy Envoy.

Conclusioni

Il lavoro di questa tesi ha avuto come obiettivo principale l'analisi comparativa tra Istio e NSM, due sistemi di Service Mesh che affrontano in modo differente la gestione delle comunicazioni tra microservizi in ambienti Cloud Native. Queste due tecnologie sono state scelte per questo lavoro perché sono molto popolari nei contesti Kubernetes ma il loro modo di operare è differente.
Dallo studio è emerso che entrambe le tecnologie rispondano a esigenze comuni quali sicurezza, osservabilità e controllo della salute del sistema, ma con approcci architetturali e filosofie progettuali profondamente diverse.
La principale differenza tra i due approcci sta nel Livello OSI in cui lavorano. Istio lavora a Livello 4 e 7, perciò si occupa di gestire richieste HTTP, gRPC e TCP e fornisce funzionalità di gestione del carico e regole sul traffico. Il modello sidecar proxy, con proxy Envoy, è facile da implementare e configurare.
NSM lavora invece a Livello 2 e 3 e quindi non implementa regole sul traffico e non fa ritrasmissioni. A differenza di Istio non si serve del CNI del cluster, ma ha dei propri modelli per configurare la connessione tra Pod. È questa la grande differenza con Istio: NSM offre la possibilità di instaurare una connessione tra Pod, indipendentemente da dove questi si trovano. NSM è perciò ideale per integrare sistemi legacy in un cluster. Anche con Istio è possibile far interagire un cluster con un sistema legacy, a patto che questo sia compatibile con protocolli HTTP e TCP. Istio utilizza funzioni di connettività già presenti nel sistema legacy, non ne implementa di nuove come fa NSM.

Nella seconda parte di questo lavoro sono stati effettuati dei test per confrontare le due tecnologie e il cluster Kubernetes semplice in ambiti di utilizzo comuni. Per fare i test sono state predisposte quattro configurazioni: un cluster K3s semplice, un cluster con Istio, una connessione vl3 di NSM tra due Pod, una connessione k2k di NSM tra due Pod.
I test sono stati fatti con l'obiettivo di misurare le prestazioni delle quattro configurazioni e il loro consumo di risorse. Per ogni configurazione sono state testate richieste HTTP, TCP e gRPC tra due Pod, poi è stato simulato un traffico variabile per osservare la variazione della latenza nelle risposte. Infine, è stato simulato il fallimento del Data Plane di Istio e NSM per vedere come hanno funzionato i meccanismi di Health Check e Healing.
Questi test sono stati scelti per valutare il funzionamento delle due tecnologie in situazioni simili a quelle che si possono presentare nella realtà. Infatti, il protocollo più utilizzato in Kubernetes è TCP (con HTTP e gRPC), il carico a cui sono sottoposti i Pod può variare in base a fattori come orario o eventi

straordinari, le risorse del cluster possono essere scarse e i componenti che forniscono connettività tra Pod possono fallire.

In conclusione, si può dire che Istio è una tecnologia matura e dalle funzionalità concrete. Le sue prestazioni nei test svolti sono state a volte inferiori a quelle di NSM, ma ha garantito una migliore affidabilità nel suo utilizzo, considerando il fatto che consuma generalmente meno risorse, offre funzionalità di gestione del traffico, fa ritrasmissione dei pacchetti persi e gestisce in modo efficace aumenti di traffico verso i suoi servizi.

NSM, d'altra parte, offre funzionalità di connettività ben superiori a quelle di Istio, soprattutto nel caso di integrazione di sistemi legacy in un cluster. In diversi dei test svolti ha offerto migliori prestazioni rispetto a Istio, ma spesso a fronte di un ampio consumo di risorse oppure di poca affidabilità. Con NSM non si può personalizzare la gestione del traffico, che può sovraccaricare i servizi nel caso di grandi quantità di richieste.

Le architetture a microservizi sono oggi molto popolari in contesti di produzione e le infrastrutture Service Mesh non possono che continuare a diffondersi. Sistemi di Service Mesh classici come Istio sono già da tempo soluzioni efficaci e semplici. Sono tante le realtà che li utilizzano, quali grandi aziende tech, servizi di e-commerce, servizi finanziari, team di DevOps.
La tecnologia NSM invece, che propone un servizio di Service Mesh differente e più orientato alla rete, non è ancora così diffuso. Il progetto non è maturo a sufficienza per offrire una esperienza d'uso facile ed affidabile. I casi d'uso di NSM sono però ricercati e specifici: l'integrazione di sistemi legacy nel cloud rimarrà ancora una necessità nei prossimi anni, visto che spesso sostituire quest'ultimi richiede ampie risorse economiche e competenze logistiche. In Italia, per esempio, le tecnologie utilizzate nelle aziende e nella pubblica amministrazione sono spesso obsolete e sono ancora diffusi sistemi che mal si integrano con il cloud.

Il lavoro di questa tesi si è concentrato su test pratici con le tecnologie Istio e NSM e sono stati fatti su un computer dalle risorse limitate. Sarebbe perciò interessante, come sviluppo futuro di questo lavoro, ripetere gli stessi test in un ambiente con risorse maggiori. Per esempio, si potrebbe simulare la comunicazione tra più Pod che risiedono in nodi fisici diversi (e non più macchine virtuali), e magari in cluster diversi. Inoltre, potrebbe essere utile approfondire il funzionamento dei Data Plane utilizzati da Istio e NSM. Per esempio, si potrebbe confrontare il funzionamento di Envoy e VPP e la loro architettura.

Bibliografia

[1] Peter Mell, Timothy Grance, *The NIST Definition of Cloud Computing*. NIST, Special Publication 800-145, 2011.

[2] https://www.redhat.com/it/topics/containers/containers-vs-vms

[3] Sensini Serena, *Docker. Sviluppare e rilasciare software tramite container*. Apogeo, 2020

[4] Binildas Christudas, *Java Microservices and Containers in the Cloud*. Apress, 2024

[5] John Kindervag, *No more chewy centers: Introducing the Zero Trust Model of Information Security*. Forrester, 2010

[6] Daniel Feldman, Emily Fox, Evan Gilman, Ian Haken, Frederick Kautz, Umair Khan, Max Lambrecht, Brandon Lum, Agustín Martínez Fayó, Eli Nesterov, Andres Vega, Michael Wardrop, *Solving the bottom turtle: a SPIFFE way to establish trust in your infrastructure via universal identity*. Book Sprints, 2020

[7] E. Gerck, *Overview of Certification Systems: X.509, CA, PGP and SKIP*. MCG - Meta-Certificate Group, 1998

[8] Michael Jones, John Bradley, and Nat Sakimura, *JSON Web Token (JWT)*. IETF, 2015

[9] Brendan Burns, Eddie Villalba, Dave Strebel, Lachlan Evenson, *Kubernetes Best Practices: Blueprints for Building Successful Applications on Kubernetes*. O'Reilly, 2019

[10] George Miranda, *The Service Mesh*. O'Reilly, 2018

[11] https://www.envoyproxy.io/docs

[12] Lee Calcote, Zack Butcher, *Istio: Up and Running: Using a Service Mesh to Connect, Secure, Control, and Observe*. O'Really, 2019

[13] Julien Pivotto, Brian Brazil, *Prometheus: Up & Running: Infrastructure and Application Performance Monitoring*. O'Really, 2023

[14] https://networkservicemesh.io/docs

[15] David Jansen, Lukas Krattiger, Shyam Kapadia, *Building Data Centers with VXLAN BGP EVPN: A Cisco NX-OS Perspective*. Ciscopress, 2017

[16] David Barach, Leonardo Linguaglossa, Damjan Marion, Pierre Pfister, Salvatore Pontarelli, Dario Rossi, *High-Speed Software Data Plane via Vectorized Packet Processing*. IEEE Communications Magazine, 2018

[17] Ted Young, Austin Parker, *Learning OpenTelemetry: Setting Up and Operating a Modern Observability System*. O'Reilly, 2024

[18] https://docs.k3s.io/